

Contents

NumPy Tutorial	1
INTRO	1
PART 1. Introduction to NumPy	3
1.1 Python Basics	3
1.2 Using Google Colab for This Tutorial	7
1.3 Basic Structure of Codes Used in This Tutorial	8
1.4 NumPy Arrays	11
1.5 Shape of Numpy Arrays	12
1.6 Accessing NumPy Arrays	17
1.7 Operations on NumPy Arrays	21
1.8 Linear Algebra	33
1.9 Saving and Loading Data with <code>.npy</code> and <code>.npz</code> Files	56
PART 2. Introduction to Assert Statements and Testing	58
PART 3. Debugging Your Code	62
3.1 Errors in Matrix Multiplication	62
Wrap-Up and Next Steps	64

NumPy Tutorial

Version 1.0 - February, 2026. Monterrey

Author: Flavio F. Contreras-Torres. Tecnológico de Monterrey.

License MIT
License CC BY 4.0

version v1.0

INTRO

This notebook is a **hands-on introduction** to numerical computing with NumPy and to some essential programming practices in Python. It is designed for beginners and for learners with prior experience who want to refresh or strengthen their understanding in how to work with arrays, perform basic linear algebra operations, and write more reliable code.

The notebook is divided into three main parts:

- **Part 1: Introduction to NumPy.** You will learn how to create and manipulate NumPy arrays, and how to use them for tasks such as vector and matrix operations, linear algebra, and polynomial multiplication.

- **Part 2: Introduction to Assert Statements and Testing.** You will learn how to use assert statements to test your code, check that your functions work as expected, and catch errors early.
- **Part 3: Debugging Your Code.** You will learn basic strategies for finding and fixing bugs in your programs, and how to use error messages and test cases to improve your code.

How to use this tutorial

This tutorial is designed to be completed in approximately **6 to 7 hours**, depending on your background and how much time you spend experimenting with the code and solving the exercises.

To reach the full training goal, you are encouraged to work through the material step-by-step. The practical core of this tutorial is **composed of two essential workbooks** that you will run directly in **Google Colab**:

1. `numpy_python_basics.ipynb`: A foundational review to ensure your Python skills are ready for data science.
2. `numpy_tutorial_exercises.ipynb`: A deep dive into specialized NumPy operations and array logic.

Submission and Grading: Please note that these two notebooks will be evaluated as **official assignments**. Once you have completed all the exercises, you must submit your final versions through the **Canvas** platform for grading.

By using Google Colab, you can run these files in your browser **without any local setup**. Read the explanations, run the code cells, and try to modify the examples to see what happens. When you reach an exercise, take your time to think about the problem and attempt a solution before looking at any hints or solutions.

You are expected to **solve these exercises yourself** by writing Python and NumPy code. The explanations provided earlier will give you the tools you need, but the real learning happens when you **try, test, make mistakes, and fix them**.

If you get stuck, use the testing and debugging sections to help you understand what went wrong rather than skipping ahead. The goal is not just to finish the notebook, but to build confidence in using NumPy and in writing, testing, and debugging your own Python code.

Sources and Learning Materials

This tutorial is not meant to be the first or the last resource you will use to learn NumPy and scientific computing in Python. Instead, it is a **curated learning path** built from several tutorials, lecture notes, exercises, and official documentation.

Many of the ideas, examples, and exercises presented here are **inspired by or adapted from** existing educational materials, including the NumPy documentation and common teaching resources used in courses and online tutorials. For this reason, you may notice that some exercises or examples look **similar to ones you have seen elsewhere**. This is intentional: these problems are standard, well-tested ways of learning core concepts.

The goal of this tutorial is not to present completely new material, but to **organize and connect** these concepts in a coherent, progressive way, with explanations, practice exercises, testing, and debugging techniques all in one place.

You are encouraged to complement this tutorial with other resources, such as:

- The official [NumPy documentation](#)
- Free course notes, books, and lecture materials [Mathematics for Machine Learning](#), [Wolfram MathWorld](#)
- Online tutorials [3blue1brown](#)

Learning works best when you see the same ideas explained in **multiple ways and from multiple sources**.

PART 1. Introduction to NumPy

[NumPy](#) and [SciPy](#) are two core libraries for scientific computing in Python, but they serve different roles. In particular, NumPy (**Numerical Python**) is the core library for numerical and scientific computing in Python. It provides the fundamental data structure *the array* and the basic numerical operations needed to work efficiently with numerical data. This way, Numpy provides an efficient multidimensional array structure that enables fast, vectorized numerical operations, forming the foundation of most scientific and data-driven workflows in the Python ecosystem. In addition, NumPy is built for performance: it allows large datasets to be processed without explicit Python loops, achieving speeds comparable to compiled languages such as C and Fortran. For this reason, NumPy also serves as a foundation for many other libraries, such as pandas (data analysis), scikit-learn (machine learning), and frameworks like TensorFlow and PyTorch, which rely on NumPy-style arrays for their internal computations.

On the other hand, SciPy (**Scientific Python**) is built on top of NumPy and extends it with a large collection of ready-to-use scientific algorithms, including tools for optimization, numerical integration, statistics, signal processing, and linear algebra. Because SciPy operates directly on NumPy arrays, it is essential to understand NumPy first before using SciPy effectively.

In this tutorial, we will focus exclusively on **NumPy** and use it in the simplest possible way, without relying on higher-level libraries such as pandas, scikit-learn, TensorFlow, or PyTorch. This approach allows us to concentrate on the core concepts—arrays, basic operations, and numerical computing—so you can build a solid foundation before moving on to more advanced tools.

In particular, this tutorial will focus on the basic concepts of **linear algebra**, such as vectors, matrices, and the operations between them. In this context, NumPy provides reliable implementations of common matrix operations, as well as routines for eigenvalue and eigenvector calculations, covariance matrices, and affine transformations. These tools are widely used in areas such as machine learning, dimensionality reduction, and mathematical modeling.

The documentation of **NumPy** is extensively referenced and is available at the official [webpage](#).

1.1 Python Basics

The tokens `+`, `-`, and `*`, and the use of parentheses for grouping, mean in Python what they mean in mathematics. The asterisk (`*`) is the token for multiplication, and `**` is the token for exponentiation. Addition, subtraction, multiplication, and exponentiation all do what you expect.

The Python interpreter can compute new values with function calls. There you might define a function `a` or `f` by specifying how it transforms an input into an output, for instance, $f(x) = 3x + 2$. Functions are like factories that take in some material, do some operation, and then send out the resulting object.

Additionally, if we want to see the results of the computation, the program needs to specify that with the function `print()`.

```
1 # Step 1. Define input data
2 a = 20 + 30
3 b = 5 ** 2
4 c = (5 + 9) * (15 - 7)
5 d = 16 - 2 * 5 // 3 + 1
6
7 # Step 2. Show the results
8 print(a)
9 print(a+b)
10 print(a-b+c)
11 print(d)
```

```
1 52
2 77
3 139
4 14
```

Remember that when more than one operator appears in an expression, the order of evaluation depends on the rules of precedence. Python follows the same precedence rules for its mathematical operators that mathematics does. For the last example, there is:

```
1 print((16 - ((2 * 5) // 3)) + 1)
```

```
1 14
```

In Python 3, the division operator `/` produces a floating point result (*even if the result is an integer*; $4/2$ is 2.0). If you want truncated division, which ignores the remainder, you can use the `//` operator (for example, $5//2$ is 2).

```
1 # Step 1. Define input data
2 a = 9 / 5
3 b = 5 / 9
4 c = 9 // 5
5 d = 18.0 // 4
6
7 # Step 2. Show the results
8 print(a)
9 print(b)
10 print(c)
```

```
1 1.8
2 0.5555555555555556
3 1
4 4.0
```

The modulus operator, sometimes also called the remainder operator or integer remainder operator works on integers (and integer expressions) and yields the remainder when the first operand is divided by the second. In Python, the modulus operator is a percent sign (%). The syntax is the same as for other operators.

```
1 # Step 1. Define input data
2 a = 7 // 3
3 b = 7 % 3
4
5 # Step 2. Show the results
6 print(a)
7 print(b)
```

```
1 2
2 1
```

```
1 print(square(3))
2 print(sub(6, 4))
3 print(sub(5, 9))
4 print(square(3) + 2)
5 print(sub(square(3), square(1+1)))
```

```
1 9
2 2
3 -4
4 11
5 5
```

Notice that when a function takes more than one input parameter, the inputs are separated by a comma. However, when you type a large integer, you might be tempted to use commas between groups of three digits, as in 42,000. This is **not a legal** integer in Python, but it does mean something else, which is legal:

```
1 print(42500)
2 print(42,500)
```

```
1 42500
2 42 500
```

Remember not to put commas or spaces in your integers, no matter how big they are. Additionally, if you are not sure what class (data type) a value falls into, Python has a function called `type` which can tell you.

```
1 print(type("Hello, World!"))
2 print("Hello, World")
3 print(type(17))
4 print(type(3.2))
5 print(type("17"))
6 print(type("3.2"))
```

```
1 <class 'str'>
2 Hello, World
3 <class 'int'>
4 <class 'float'>
5 <class 'str'>
6 <class 'str'>
```

Notice that strings in Python can be enclosed in either single quotes (') or double quotes ("), or three of each (''' or """)

```
1 print(type('This is a string.'))
2 print(type("And so is this."))
3 print(type("""and this."""))
4 print(type(''and even this...'))
```

```
1 <class 'str'>
2 <class 'str'>
3 <class 'str'>
4 <class 'str'>
```

Triple quoted strings can even span multiple lines, such as:

```
1 print("""This message will span
2 several lines
3 of the text.""")
```

```
1 This message will span
2 several lines
3 of the text.
```

Sometimes it is necessary to convert values from one type to another. Python provides a few simple functions that will allow us to do that. The functions `int`, `float` and `str` will convert their arguments into types `int`, `float` and `str` respectively.

The `int` function can take a floating point number or a string, and turn it into an `int`. For example:

```
1 print(3.14, int(3.14))
2 print(3.0, int(3.0))
3 print("2345", int("2345"))
```

```
1 3.14 3
2 3.0 3
3 2345 2345
```

The type converter `float` can turn an integer, a float, or a syntactically legal string into a float:

```
1 print(float("123.45"))
2 print(type(float("123.45")))
```

```
1 123.45
2 <class 'float'>
```

The type converter `str` turns its argument into a string. Remember that when we print a string, the quotes are removed in the output window. However, if we print the type, we can see that it is definitely `str`.

```
1 print(str(17))
2 print(str(123.45))
3 print(type(str(123.45)))
```

```
1 17
2 123.45
3 <class 'str'>
```

1.2 Using Google Colab for This Tutorial

If you’ve ever wanted to write code without the headache of software installation, **Google Colab** is the perfect place to start. It’s a cloud-based platform that lets you execute Python directly in your browser. We’ll use it throughout this tutorial so you can focus entirely on mastering NumPy instead of troubleshooting your computer setup. Think of it as **Google Docs, but for programmers**.

What is Google Colab?

Beyond just avoiding installation, Google Colab provides an interactive environment called a **Notebook**. These notebooks are powerful because they allow you to combine:

- Executable Code: Write and run Python instantly.
- Rich Documentation: Add explanations, notes, and instructions using Markdown.
- Visuals: Embed images, HTML, and even complex mathematical formulas using LaTeX.

Because it’s integrated with the Google ecosystem, your progress saves automatically to **Google Drive**. You can share your work as easily as a link, allowing others to view, comment, or collaborate in real-time. All you need is a browser and an internet connection to get started.

Why Use Colab?

Since we’ve already seen that Colab is cloud-based and requires zero setup, here are the specific reasons it’s the best choice for learning **NumPy**:

- **Pre-installed Libraries:** You don’t need to worry about pip install. Essential libraries like NumPy, Pandas, and Matplotlib are already configured and ready to use.
- **Google’s Computing Power:** Your code runs on Google’s high-performance servers. This means your computer’s battery and RAM stay fresh while the cloud handles the heavy lifting.
- **Hardware Acceleration:** If our data tasks get complex, you can toggle on a GPU for free to speed up calculations—something that usually requires expensive hardware.
- **Cell-Based Learning:** Unlike a traditional script, the Interactive Format allows you to run code in small “cells.” You can test one line of NumPy at a time, see the result immediately, and move on.

- **Seamless Collaboration:** Just like any other Google Workspace tool, you can share your notebooks with a link to get help or show off your progress.

Colab and Jupyter Notebooks

If you’ve heard of **Jupyter Notebooks**, you’re already one step ahead. Google Colab is essentially a cloud-hosted version of the Jupyter technology.

- **Same DNA:** Colab uses the standard .ipynb file format. This means any notebook you create here can be downloaded and opened in a local Jupyter environment later.
- **The Main Difference:** While traditional Jupyter requires you to manage your own Python “environments” and local installations, Colab provides a pre-configured workspace on Google’s infrastructure.
- **Why start here?** We are using Colab today to remove technical barriers, allowing you to focus entirely on NumPy logic.

In **Table 1**, a comparison is shown between these two environments to help you decide which setup best suits your future projects.

Table 1: Key Differences Between Google Colab and Local Jupyter Notebooks

Feature	Google Colab	Jupyter Notebook
Setup	No installation required (cloud-based)	Manual Python and Jupyter installation (local)
Internet	Required connection	Works offline
Hardware	Uses Google’s servers	Uses your own computer
Environment	Pre-installed popular libraries	Customizable environments
Collaboration	Live sharing and commenting (like Google Docs)	Shared via files (*.ipynb)

Looking Ahead: In future tutorials, we will explore how to run Jupyter locally on your own machine. This will give you more control over offline access and custom configurations as your projects become more advanced. For now, Colab is the fastest way to get your hands on the code.

In this first practical session, we will be using the notebook `numpy_python_basics.ipynb`.

Our goal is to review the core concepts from **Subsection 1.1: Python Basics**. While you may have seen these concepts before, running them inside Google Colab will help you get comfortable with the interactive “cell” environment before we dive deep into NumPy.

1.3 Basic Structure of Codes Used in This Tutorial

In this tutorial, all code examples follow a simple and consistent structure. First, we import the required **libraries**. To work with numerical data in Python, we first import the **NumPy** library. By convention, NumPy is imported using the abbreviation `np`, which makes the code shorter and easier to read.

```
1 import numpy as np      # standar NumPy import
```


After importing NumPy as `np`, we access its tools by writing `np.` followed by the name of a function or object. Then, we define the input data, usually as NumPy arrays. For example, in the next section we will see how to create arrays using the function call `np.array()`. Thus, the resulting **array** can be stored in a variable by assigning it a name, for instance, we can choose the letter `a` to refer to the array in `a = np.array(...)`.

```
1 import numpy as np      # required
2
3 x = [1,2,3]             # a Python list with some elements
4 a = np.array(x)         # convert the list into a NumPy array and store it as 'a'
```

In the next example, **addition** is already defined for NumPy arrays, so we just use the operator `+`.

```
1 import numpy as np      # required
2
3 # Step 1. Define input data
4 a = np.array([1, 2, 3])
5 b = np.array([4, 5, 6])
6
7 # Step 2. Perform an operation (already defined in NumPy)
8 c = a + b
9
10 # Step 3. Show the result
11 print(c)
```

Sometimes the mathematical expression we need is already available through existing operations, while in other cases we will write the equation explicitly in code. For instance, the equation $F = m \cdot a$ is not a built-in physics function, so we write it ourselves using the operator `*`.

Additionally, in many cases, we also define a function using `def`, which groups a set of operations under a name so it can be reused. Inside the function, we perform the necessary computations and use `return` to send back the result.

```
1 import numpy as np      # required
2
3 # Step 1. Define a function
4 def compute_force(mass, acceleration):
5     """
6     Compute the force using Newton's second law:  $F = m \cdot a$ .
7
8     Arguments:
9     mass : number or numpy.ndarray
10         The mass of the object(s).
11     acceleration : number or numpy.ndarray
12         The acceleration of the object(s).
13
14     Returns:
15     force : number or numpy.ndarray
16         The computed force.
17     """
18     force = mass * acceleration
19     return force
```

```
20
21 # Step 2. Define input data
22 m = np.array([1.0, 2.0, 3.0]) # mass
23 a = np.array([9.8, 9.8, 9.8]) # acceleration
24
25 # Step 3. Call the function
26 F = compute_force(m, a)
27
28 # Step 4. Show the result
29 print(F)
```

Note: When defining functions, it is important to include short comments or documentation strings (written between triple quotes `""" ... """`) that explain what the function does, what arguments it expects, and what it returns. These docstrings make the code easier to read, understand, and maintain, especially when functions are reused or shared with others.

Finally, the same idea, but with matrices (linear algebra style). In this case, we use the operator `@`.

```
1 import numpy as np # required
2
3 # Step 1. Define a function for matrix-vector multiplication
4 def apply_matrix(A, x):
5     """
6     Apply a matrix to a vector using matrix-vector multiplication.
7
8     Arguments:
9     A : numpy.ndarray
10         A two-dimensional matrix.
11     x : numpy.ndarray
12         A one-dimensional vector.
13
14     Returns:
15     y : numpy.ndarray
16         The resulting vector after multiplication.
17     """
18     y = A @ x
19     return y
20
21 # Step 2. Define input data
22 A = np.array([[1, 2],
23               [3, 4]])
24 x = np.array([1, 1])
25
26 # Step 3. Compute result
27 y = apply_matrix(A, x)
28
29 # Step 4. Show the result
30 print(y)
```

As you can see, **arrays** appear in all the examples, because they are the basic data structure used to represent vectors, matrices, and numerical data in NumPy. For this reason, before moving on to more advanced

topics, we need to understand how arrays are created, how they are shaped, and how operations are applied to them.

1.4 NumPy Arrays

A NumPy **array** is a data structure used to store numbers in an organized, grid-like form (such as a list, table, or matrix) so they can be processed efficiently.

Unlike Python lists, NumPy arrays store **elements of the same type** and allow fast mathematical operations on all values at once, which makes them ideal for numerical computing and scientific applications.

New arrays can be created in several ways. One simple method is to start from a Python list (a basic collection of values) and convert it into a NumPy array:

```
1 import numpy as np      # required; omitted in the following exercises
2
3 a = np.array([1,2,3])    # This is a one-dimensional array
4 print(a)
```

```
1 [1 2 3]
```

Note: `np.array` is a **function** in NumPy. The parentheses () are used to call this function, while the square brackets [] define the data being passed to it.

```
1 b = np.array([[1, 2, 3],    # This is a two-dimensional array
2               [4, 5, 6],
3               [7, 8, 9]])
4 print(b)
```

```
1 [[1 2 3]
2  [4 5 6]
3  [7 8 9]]
```

Note: For 2-dimensional Numpy arrays, all rows must have the same number of elements.

NumPy arrays can be created from both Python **lists** and **tuples**. A list [1, 2, 3] and a tuple (1, 2, 3) differ in Python because lists can be modified while tuples cannot. For instance, a **list** can be a shopping list, a to-do list of tasks, the name of students in a class, books you plan to read in this year. On the other hand, examples of **tuples** can be the days of the week, the months of the year, the coordinates of a point (x,y,z), a person's date of birth (day, month, year), the dimension of a rectangle (width, height).

However, when either one is passed to `np.array()`, NumPy copies the values into a **new NumPy array**. After this conversion, the original list or tuple is no longer relevant, and the resulting NumPy array behaves exactly the same in both cases. In other words, the difference matters before creating the NumPy array (list vs tuple), but not after, because NumPy creates its own data structure.

```
1 lst = [1, 2, 3]          # Create a NumPy array from a list
2 a = np.array(lst)
3
```

```
4  tp1 = (1, 2, 3)      # Create a NumPy array from a tuple
5  b = np.array(tp1)
6
7
8  print(a)
9  print(b)
```

```
1  [1 2 3]
2  [1 2 3]
```

The outputs are the printed representation of NumPy arrays.

In NumPy, every array has a data type, called dtype, which tells NumPy what kind of numbers the array stores.

```
1  a = np.array([1, 2, 3])
2  b = np.array([1.0, 2.0, 3.0])
3
4  print("Type of array in a:", a.dtype)
5  print("Type of array in b:", b.dtype)
```

```
1  Type of array in a: int64
2
3  Type of array: float64
```

Note: NumPy arrays have a dtype attribute that controls the type of numbers they store (int, float, complex, etc.). In more advanced code, you may want to match the data type of different arrays explicitly.

In NumPy, we can use `np.array()` to create vectors and `np.dot()` to compute their dot product.

```
1  def length(x):
2      """Compute the length of a vector"""
3      length_x = np.sqrt(np.dot(x, x)) # using the relationship: ||x|| = sqrt(x·x)
4
5      return length_x
6
7  print(length(np.array([1,0])))
```

```
1  1.0
```

1.5 Shape of Numpy Arrays

The **shape** of a NumPy array describes how many elements it has and how they are arranged. You can use the function `np.shape` to obtain the shape of a Numpy array.

The shape of NumPy arrays is printed as a tuple of numbers, where each number represents the size of one dimension. For example, a shape of `(3,)` means the array has 3 elements in one dimension. The comma is important because it tells Python that this is a tuple, not just a number.

```
1 a = np.array([1, 2, 3])
2 print(a.shape)
```

```
1 (3,)
```

Note: Remember that tuples (writing always using parenthesis) are similar to lists, but they cannot be changed after they are created (i.e. you cannot add, remove, or replace elements). This property makes tuples useful for representing fixed information, such as the shape of a NumPy array.

```
1 b = np.array([[1], [2], [3]])
2 print(b.shape)
```

```
1 (3, 1)
```

For this example, the tuple (3,1) represents the **shape** of the NumPy array, meaning the array has 3 rows and 1 column. For two dimensional arrays, you can consider the first element of the tuple to be the number of rows and the second element to be the number of columns.

```
1 c = np.array([[1, 2],
2               [3, 4],
3               [5, 6]])
4 print(c.shape)
```

```
1 (3, 2)
```

For this example, the tuple (3,2) represents the **shape** of 3 rows and 2 columns. Note that the shape of an array is determined by how values are grouped with brackets, not by line breaks. Line breaks only improve readability and do not affect the array's structure.

As indicated, when working with NumPy arrays, `b.shape` returns a tuple describing dimensions:

```
1 b = np.array([1, 2, 3, 4, 5])
2 D = b.shape
3 print(D)
```

```
1 (5,)
```

However, to extract just the dimension value from this **1-element tuple**, we use **tuple unpacking** with a comma:

```
1 b = np.array([1, 2, 3, 4, 5])
2 D, = b.shape
3 print(D)
```

```
1 5
```

This is a Python idiom that's very common in scientific computing code. It makes the intention clear: "I'm expecting a 1-element tuple and I want that single value", in other words, the comma in `D`, signals for taking the single element from this tuple.

If A is a matrix (a two-dimensional NumPy array), the expression

```
1 n = A.shape[0]
```

means that we store the numbers of rows of A in a variable called n . Here, `A.shape` returns the size of the matrix as a pair:

```
1 (number of rows, number of columns)
```

Therefore, `A.shape[0]` selects the **first value** of that pair, which corresponds to the number of **rows**. Similarly, `A.shape[1]` selects the second value, which correspond to the number columns.

```
1 d = np.array([[1, 2, 3],
2               [4, 5, 6],
3               [7, 8, 9]])
4
5 print(d.shape)
6 print(d.shape[0])
7 print(d.shape[1])
```

```
1 (3, 3)
2 3
3 3
```

For this example, the tuple `(3, 3)` represents the **shape** of 3 rows and 3 columns.

In a **matrix**, each element is identified by two indices: the **row index** and the **column index**. The principal diagonal runs from the top-left corner to the bottom-right corner of the matrix.

Along this diagonal, each element lies in the same row and column position. This means that the row index and the column index are equal. For this reason, every element on the principal diagonal has an index of the form (i, i) , such as $(0, 0)$, $(1, 1)$, and so on.

This indexing pattern is what allows diagonal elements to be accessed easily in code using expressions like `A[i, i]`.

Creating Numpy arrays based on shape

NumPy allows you to create arrays by specifying their shape, **without providing the individual values explicitly**. This is useful when you need arrays of a certain size initialized with default values.

Common examples include arrays filled with **zeros**, **ones**, or **random numbers**, where the shape defines the number of rows and columns. You can use the functions `np.zeros` and `np.ones` to create Numpy arrays of a specified shape.

```
1 a = np.zeros((5,))
2 print(a)
```

```
1 [0. 0. 0. 0. 0.]
```

```
1 b = np.zeros((2, 3))
2 print(b)
```

```
1 [[0. 0. 0.]
2  [0. 0. 0.]]
```

```
1 c = np.ones((3, 2))
2 print(c)
```

```
1 [[1. 1.]
2  [1. 1.]
3  [1. 1.]]
```

Note: Remember that the input for these functions represent the shape of the Numpy arrays which you want to produce.

Reshaping arrays

Reshaping an array means changing its shape (dimensions) **without changing its data**. Reshaping is useful when you want to reorganize the same data into a different structure, such as converting a one-dimensional array into a matrix or modifying the number of rows and columns. A common use case is transforming one-dimensional NumPy arrays into two-dimensional row vectors or column vectors.

NumPy allows this using the function `np.reshape` to change the shape of a Numpy array.

Reshape is often used in conjunction with the function `np.arange`. The function `np.arange(x)` returns a one-dimensional array containing the values $[0, 1, \dots, x - 1]$.

To reshape an array, you specify the desired shape as a tuple. One value in the tuple can be set to -1, which tells NumPy to automatically determine the appropriate size for that dimension. An error will occur if more than one value is set to -1.

```
1 a = np.arange(3)           # Step 1: Create a one-dimensional NumPy array
2
3 print("Array a:")
4 print(a)
5
6 print("Shape of a:")
7 print(a.shape)
```

```
1 Array a:
2 [0 1 2]
3 Shape of a:
4 (3,)
```

The sequence starts at **0** because Python (and NumPy) use **zero-based indexing**. This means that counting begins at zero instead of one. Starting from zero makes it easier for the computer to keep track of positions and lengths of data. So, in this example, **0** is the first position, **1** is the second position, and **2** is the third position. Together, these numbers represent the first three positions in the array.

```
1 a = a.reshape((1, -1))      # Step 2: Reshaping it to a row vector
2
3 print("Reshaped array a:")
4 print(a)
5
6 print("New shape of a:")
7 print(a.shape)
```

```
1 Reshaped array a:
2 [[0 1 2]]
3 New shape of a:
4 (1, 3)
```

In the shape for the first output, (3,) the value 3 indicates the number of elements in the single dimension of the array. For the second output, the first value of the tuple is 1 because a row vector only has 1 row.

```
1 a = np.arange(3)
2 a = a.reshape((-1, 1))      # Reshaping it to a column vector
3
4 print("Reshaped array a:")
5 print(a)
6 print("New shape of a:")
7 print(a.shape)
```

```
1 Reshaped array a:
2 [[0]
3  [1]
4  [2]]
5 New shape of a:
6 (3, 1)
```

These two operations can be combined into a single line, since the `reshape()` method can be applied directly to the array created by `np.arange()`.

```
1 a = np.arange(3).reshape((-1, 1))
```

This produces the same column vector with shape (3, 1).

Note: Remember that the operations are read from **left to right**. In the last example, `np.arange(3)` creates the array, and then `reshape((-1, 1))` is applied to that result. So, in general, chained NumPy operations are applied step by step, from left to right, with each operation acting on the result of the previous one.

```
1 a = np.arange(6)
2 print("A: Original")
3 print(a)
4 print(a.shape)
5
6 a = a.reshape((3, 2))
7 print("\nA: Reshaped\n")
8 print(a)
9 print("\n", a.shape)
```

```
1 A: Original
2 [0 1 2 3 4 5]
3 (6,)
4
5 A: Reshaped
6
7 [[0 1]
8  [2 3]
9  [4 5]]
10
11 (3, 2)
```

Note: Observe that the newline character `\n` (for instance, `print("\nA: Reshaped\n")`) is used to insert **line breaks** in printed output. It allows you to add blank lines to separate sections and improve readability without changing the content of the data being printed.

1.6 Accessing NumPy Arrays

Once we have a NumPy array, we often want to look at or use **individual values** inside it. This is called accessing an array. We already know that NumPy arrays are 0 indexed and each value in a NumPy array has a position, called an **index**. In Python, counting starts at zero, so the first value is at position 0, the second at position 1, and so on.

To access a value (i.e., element of a NumPy array), we write the name of the array followed by the index inside **square brackets** `[ ]`. If A is a one dimensional array, then you can use `A[i]` to access the i^{th} element of A . If A is a two dimensional array, then you can use `A[i, j]` to access element $a_{i,j}$, where i is the row and j is the column.

```
1 a = np.arange(4)
2
3 print(a)
4
5 print("The 2th element of A")
6 print(a[2])
```

```
1 [0 1 2 3]
2 The 2th element of A
3 2
```

```
1 a = np.arange(9).reshape(3, 3)
2
3 print(a)
4
5 print("\nThe 1th row of the array:")
6 print(a[1])
7
8 print("\nElement at 0th row and 0th column:")
9 print(a[0, 0])
10
11 print("\nElement at 2th row and 1th column:")
12 print(a[2, 1])
```

```
1 [[0 1 2]
2  [3 4 5]
3  [6 7 8]]
4
5 The 1th row of the array:
6 [3 4 5]
7
8 Element at 0th row and 0th column:
9 0
10
11 Element at 2th row and 1th column:
12 7
```

Array slicing

Array **slicing** means selecting a part of an array instead of just one single value. It allows us to take several values at once by specifying a range of positions. Instead of asking for *one position*, slicing asks for a *range of positions*. Think of slicing like cutting a piece of cake: you choose where to start cutting and where to stop, so the slice includes everything in between.

Read more about the slicing notation in the [documentation](#).

Slicing uses the format:

```
1 start : end
```

Here, start indicates where to begin (included), while end shows where to stop (not included).

Suppose *A* is a one dimensional array with elements at positions

```
1 Index:  0   1   2   3   4
2 Value: A[0] A[1] A[2] A[3] A[4]
```

If you want to get all elements whose index is greater than or equal to 1 but less than 4, you use the slice `A[start:end]`:

```
1 A[1:4]
```

This selects the elements at positions:

```
1 1, 2, 3
```

The element at position **4** is not included, because the end index is always excluded.

If you want to get all elements of the array, you can write: `A[:]`. This means from the beginning to the end of the array.

For a two-dimensional array *A*, you can obtain the i^{th} column using: `A[:, i]`. This notation means: take all rows (`:`) from column *i*.

The result is returned as a **one-dimensional array**, even though it comes from a column. If you need the result to be treated as a column vector, it must be reshaped explicitly.

```
1 a = np.arange(16).reshape(4, 4)
2 print(a)
3
4 print("\nThe 1th column of the array:")
5 print(a[:, 1])
6
7 print("\nThe 3th column of the array reshaped into a column vector:")
8 print(a[:, 3].reshape(-1, 1))
```

```
1 [[ 0  1  2  3]
2  [ 4  5  6  7]
3  [ 8  9 10 11]
4  [12 13 14 15]]
5
6 The 1th column of the array:
7 [1 5 9 13]
8
9 The 3th column of the array reshaped into a column vector:
10 [[ 3]
11  [ 7]
12  [11]
13  [15]]
```

A built-in Python function used to generate a sequence of numbers is `range()`. It is most commonly used when we want to repeat an action a certain number of times, especially in loops.

The number produced by `range()` start at **0** by default, increase by **1**, and stop **before** the final number.

```
1 range(stop)
```

This generates numbers starting from 0 up to, but not including, stop. For example, `range(5)` represents the list of numbers **0, 1, 2, 3, 4**.

The function `range()` can be used with arrays. If an array has length n , `range(n)` produces exactly the indices needed to access all its elements, that is, the first index 0 up to the last index $n - 1$.

```
1 a = np.arange(16).reshape(4, 4)
2 n = a.shape[0]
3
4 print(a)
5 print(range(n))
```

```
1 [[ 0  1  2  3]
2  [ 4  5  6  7]
3  [ 8  9 10 11]
4  [12 13 14 15]]
5
6 range(0, 4)
```

Note: Even though it looks like a tuple, `range(0, 4)` is **NOT** a tuple. It is an object that represents a **sequence of numbers**, namely, start at 0, stop before 4.

Exercise 1: Find the trace of a matrix

The trace of a square matrix is defined as the sum of the elements on its main diagonal (from top-left to bottom-right).

For example, for the matrix

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

the trace is:

$$\text{trace}(A) = 1 + 4 = 5$$

In this exercise, you will implement a function that computes the **trace** without using NumPy's built-in trace functions.

Hint: Each element of the principal diagonal has an index of the form (i, i) .

```
1 import numpy as np
2
3 def trace(A):
4     """
5     Compute the trace of a square matrix A
6
7     Arguments:
```

```
8     A : numpy.ndarray
9         A two dimensional square matrix
10
11     Returns:
12     s : number
13         The trace of A
14     """
15
16     # Initialize the sum of diagonal elements
17     s = 0
18
19     ### BEGIN SOLUTION
20
21     # Step 1. Obtain the number of rows in A
22     n = None          # Replace with the correct syntax to get the number of rows
23
24     # Step 2. Loop over all elements of the principal diagonal
25     for i in range(n):
26         s += None     # Replace the None with the required element of A
27
28     ### END SOLUTION
29
30     return s
```

Verify your solutions s for this exercise by computing the trace manually using indexing and loops for the following matrices A . This helps reinforce how NumPy performs matrix operations under the hood.

```
1  A = np.array([[3, 2, 7],
2                [4, 9, 0],
3                [1, 8, 5]])
4  s = 17
5
6
7  A = np.array([[12, -2, 31, 18],
8                [32, -77, -24, 19],
9                [87, 93, -53, 13],
10               [49, 81, 63, 70]])
11 s = -48
```

1.7 Operations on NumPy Arrays

NumPy does not treat arrays as single numbers. Instead, it applies the operation to each element inside the array. This means that NumPy allows you to perform mathematical operations directly on arrays. When you use operators such as `*`, `+`, `-`, `**` and `/` on NumPy arrays, the operation is applied **element by element**.

```
1  a = np.array([4, 1, 2],
2                [7, 2, 3])
3
4  b = np.array([3, 6, 9],
5                [7, 8, 2])
```

```
1 print(a + b)
```

```
1 [[ 7  7 11]
2  [14 10  5]]
```

```
1 print(a - b)
```

```
1 [[ 1 -5 -7]
2  [ 0 -6  1]]
```

```
1 print(a * b)
```

```
1 [[12  6 18]
2  [49 16  6]]
```

```
1 print(a / b)
```

```
1 [[1.33333333 0.16666667 0.22222222]
2  [1.          0.25        1.5        ]]
```

```
1 print(a ** b)
```

```
1 [[    64      1   512]
2  [823543   256     9]]
```

Comparison operators

There also exist other operators such as '==' (equal to), '!=' (not equal to), '>' (greater than), '<' (less than), '>=' (greater than or equal), '<=' (less than or equal), called comparison or relational operators. They return true or false depending on the comparison, for instance, `a != b` returns True if a and b are different, and returns False if a and b are the same.

```
1 a = np.array([1, 2, 3, 4])
2 b = np.array([1, 0, 3, 5])
3
4 print(a == b)
5 print(a != b)
6 print(a > b)
7 print(a <= b)
```

```
1 [True, False, True, False]
2 [False, True, False, True]
3 [False, True, False, False]
4 [True, False, True, True]
```

```
1 alpha = np.array([0.5, 0.0, 0.5, 0.0])
2 mask = alpha != 0 # Check each element: is it NOT equal to 0?
```

```
1 alpha = [0.5, 0.0, 0.5, 0.0]
2 Check: 0.5 != 0 → True
3     0.0 != 0 → False
4     0.5 != 0 → True
5     0.0 != 0 → False
6 mask = [True, False, True, False]
```

np.sqrt(), np.exp(), and np.log()

Some other useful functions in Numpy are [np.sqrt\(\)](#), [np.exp\(\)](#), [np.log\(\)](#) which apply the corresponding operation to every element of the inputted Numpy array.

```
1 print(np.sqrt(a))
```

```
1 [[2.          1.          1.41421356]
2  [2.64575131 1.41421356 1.73205081]]
```

```
1 print(np.exp(a))
```

```
1 [[ 54.59815003  2.71828183  7.3890561 ]
2  [1096.63315843  7.3890561  20.08553692]]
```

```
1 print(np.log(a))
```

```
1 [[1.38629436 0.          0.69314718]
2  [1.94591015 0.69314718 1.09861229]]
```

np.mean()

NumPy provides the function [np.mean\(\)](#) to compute the arithmetic mean along the specified axis.

```
1 numbers = np.array([1, 2, 3, 4, 5])
2 average = np.mean(numbers) # calculate the mean (average)
3
4 print(f"Numbers: {numbers}")
5 print(f"Mean: {average}")
```

```
1 Numbers: [1 2 3 4 5]
2 Mean: 3.0
```

In many data analysis techniques, we **center the data** by subtracting the mean of each feature. This shifts the data to have zero mean without changing its shape or relationships.

The data are centered for:

- Removes Bias: Eliminates systematic offsets so we focus on variation
- Improves Numerical Stability: Prevents large numbers that can cause computational issues
- Standard Starting Point: Many algorithms assume centered data
- Interpretability: Makes patterns easier to see and compare

```
1 data = np.array([[10, 5],
2                 [8, 4],
3                 [12, 6],
4                 [9, 5.5],
5                 [11, 5.8]])
6
7 print("Original data:")
8 print(data)
9 print(f"\nMean of x: {np.mean(data[:, 0]):.2f}")
10 print(f"Mean of y: {np.mean(data[:, 1]):.2f}")
11
12 # CENTERING: Subtract column means
13 data_normalized = data - np.mean(data, axis=0)
14
15 print("\nCentered data (data - mean):")
16 print(data_normalized)
17 print(f"\nNew mean of x: {np.mean(data_normalized[:, 0]):.2f}")
18 print(f"New mean of y: {np.mean(data_normalized[:, 1]):.2f}")
```

```
1 Original data:
2 [[10.  5. ]
3  [ 8.  4. ]
4  [12.  6. ]
5  [ 9.  5.5]
6  [11.  5.8]]
7
8 Mean of x: 10.00
9 Mean of y: 5.26
10
11 Centered data (data - mean):
12 [[ 0.   -0.26 ]
13  [-2.   -1.26 ]
14  [ 2.    0.74 ]
15  [-1.    0.24 ]
16  [ 1.    0.54 ]]
17
18 New mean of x: 0.00
19 New mean of y: 0.00
```

In this example, `axis=0` tells the computer: “For each column, look at all the numbers in that column and calculate their average.” So, `np.mean(data, axis=0)` is used to obtain one average for each column.

`np.pad()`

Sometimes, we need two NumPy arrays to have the **same length** before we can combine them in an operation. One simple way to do this is to **pad** (extend) the shorter array with extra values, such as zeros.

NumPy provides the function `np.pad()` for this purpose.

For example:

```
1 A = np.array([1, 2, 3])
2 B = np.array([4, 5])
3
4 # Pad B with one zero at the end so it has the same length as A
5 B_padded = np.pad(B, (0, 1))
6
7 print("A:", A)
8 print("B padded:", B_padded)
```

```
1 A: [1 2 3]
2 B padded: [4 5 0]
```

In this example, `(0, 1)` means: - Add 0 values at the beginning of the array - Add 1 value at the end of the array

This way, `np.pad()` allows us to extend an array with zeros (or other values) to make its size match another array. This is useful in polynomial multiplication when the two coefficient arrays do not have the same length.

`np.flip()`

Additionally, it is useful to **reverse the order** of elements in a NumPy array. For example, you may want to turn:

```
1 [1, 2, 3, 4]
```

into:

```
1 [4, 3, 1, 1]
```

NumPy provides the function `np.flip()` to do exactly this. Here is a simple example:

```
1 A = np.array([1, 2, 3, 4])
2
3 A_reversed = np.flip(A)
4
5 print("Original array:", A)
6 print("Reversed array:", A_reversed)
```

```
1 Original array: [1 2 3 4]
2 Reversed array: [4 3 2 1]
```

So, `np.flip(A)` returns a new array with the same elements as `A` but in reverse order. This function is useful in polynomial multiplication because reversing one of the coefficient arrays can help align terms correctly when computing the sums of products.

`np.where()`

It is a selection/conditional function that uses boolean arrays but doesn't create them. This function takes boolean arrays as input and returns indices or selected values as output.

```
1 np.where(condition, x, y)
```

The syntax returns such elements from `x` where `condition` is `True` and elements from `y` where `condition` is `False`.

```
1 idx = np.where(alpha != 0)[0]
```

Think of a classroom, then

- `alpha != 0` indicates which students raised their hand? -> Boolean mask
- `np.where(alpha != 0)` means what are the seat numbers of students who raised hands? -> Indices
- `[0]` is for give me just the list of seat numbers -> First element of tuple

Remember that `np.where()` does not return the mask itself but it returns where the `True` values are located in the mask

```
1 alpha = np.array([0.5, 0.0, 0.5, 0.0])
2 y = np.array([+1, +1, -1, -1])
3
4 # Method 1: Get mask, then indices
5 mask = alpha != 0
6 print("mask = alpha != 0:")
7 print(f"  mask = {mask}")
8
9 sv_idx = np.where(mask)[0]
10 print(f"\nsv_idx = np.where(mask)[0]:")
11 print(f"  sv_idx = {sv_idx}")
12
13 sv_labels = y[sv_idx]
14 print(f"\nsv_labels = y[sv_idx]:")
15 print(f"  sv_labels = {sv_labels}")
16
17 # Method 2: Direct boolean indexing (no indices needed)
18 sv_labels_direct = y[alpha != 0]
19 print(f"\nsv_labels_direct = y[alpha != 0]:")
20 print(f"  sv_labels_direct = {sv_labels_direct}")
```

```
1 mask = alpha != 0:
2     mask = [ True False  True False]
3
```

```
4 sv_idx = np.where(mask)[0]:  
5     sv_idx = [0 2]  
6  
7 sv_labels = y[sv_idx]:  
8     sv_labels = [ 1 -1]  
9  
10 sv_labels_direct = y[alpha != 0]:  
11     sv_labels_direct = [ 1 -1]
```

Broadcasting

Broadcasting is a rule that allows NumPy to perform operations between arrays of different shapes. Instead of requiring arrays to have exactly the same size, NumPy automatically adjusts the smaller array so that the operation can be carried out element-wise.

Using broadcasting, Numpy arrays that are not of the same dimension can be stretched/duplicated so that they are of the same dimension.

To know more about broadcasting, you can read the [documentation](#).

The simplest example for this is when you have to multiply all elements of a Numpy array with a scalar or add a scalar to all elements of the array. In this case, broadcasting means *use the same value multiple times so that the operation can be applied to all elements*. Imagine adding the same amount of salt to every bowl of soup. You don't need a different spoon for each bowl — you reuse the same one. Then, the regular element-wise operation is applicable.

```
1 print(a * 2)
```

```
1 [[ 8  2  4]  
2  [14  4  6]]
```

```
1 print(a + 2)
```

```
1 [[6 3 4]  
2  [9 4 5]]
```

Broadcasting lets a smaller array act as if it were expanded to match the shape of a larger array, so that element-wise operations can be performed. This way, broadcasting can be applied during operation between two arrays of different shapes. Suppose that a is an array of shape (2, 3), while c has shape (1, 3). When an operation is performed between a and c , NumPy automatically applies broadcasting to make the shapes compatible.

Conceptually, you can think of the array c as being **repeated along the row axis**, so that it behaves as if it had shape (2, 3). This allows the operation to be carried out element-wise between the two arrays. This duplication is conceptual only — NumPy does not actually copy the data in memory.

```
1 c = np.array([5, 3, 4]).reshape((1, 3))  
2
```

```
3 print('a:\n', a)
4 print('c:\n', c)
5 print('a + c:\n', a + c)
```

```
1 a:
2  [[4 1 2]
3   [7 2 3]]
4 c:
5  [[5 3 4]]
6 a + c:
7  [[ 9  4  6]
8   [12  5  7]]
```

NumPy applies broadcasting automatically. No additional function or special command is needed. When you perform an operation between arrays, NumPy checks their shapes and applies broadcasting **by default if the shapes are compatible**.

You will get an error if you try to perform an element-wise operation on two arrays whose shapes are not compatible. For element-wise operations, NumPy requires the arrays to be broadcastable, meaning their dimensions must match or follow specific broadcasting rules.

```
1 a = np.arange(6).reshape((2, 3))
2 print(a)
3
4 b = np.arange(4).reshape((2, 2))
5 print(b)
6
7 print(a + b)
```

```
1  [[0 1 2]
2   [3 4 5]]
3  [[0 1]
4   [2 3]]
5
6
7
8  -----
9
10 ValueError                                Traceback (most recent call last)
11
12 ValueError: operands could not be broadcast together with shapes (2,3) (2,2)
```

Now you will write a function which makes use of broadcasting to perform operations on a Numpy array.

Exercise 2: Is broadcasting used in the sigmoid function?

The sigmoid function:

$$\text{sigmoid}(x) = \frac{1}{1 + e^{-x}}$$

is important in machine learning because it maps any real-valued input to a value between 0 and 1. This property makes it especially useful for modeling probabilities and binary outcomes.

In practice, sigmoid is used in binary classification models, such as logistic regression and the output layer of binary neural networks. The sigmoid output represents the probability that an input belongs to the positive class. Sigmoid is mathematically paired with the binary cross-entropy (log loss) function, which provides well-behaved gradients for optimization and enables effective training through gradient descent. For this reason, sigmoid is typically used only in output layers, while other activation functions (e.g., ReLU) are preferred in hidden layers.

In this exercise, you will use NumPy broadcasting to apply the **sigmoid function** to different types of inputs. First of all, write a function which computes the sigmoid of x . Note that x might be a real number or a Numpy array.

```
1  import numpy as np
2
3  def sigmoid(x):
4      """
5      Computes the sigmoid of x
6
7      Arguments:
8      x : float or a numpy.ndarray
9          A real number or a Numpy array
10
11     Returns:
12     s : float or a numpy.ndarray
13         The sigmoid of x
14     """
15
16     ### BEGIN SOLUTION
17
18     s = None # Replace the None with the required expression for s
19
20     ### END SOLUTION
21
22     return s
```

Verify your solutions s for this exercise by computing the expected values for the following float and arrays.

```
1  x = 2.1
2  expected = 0.8909031
3
4
5  x = np.array([[3.4, -7.1, 9.4],
6               [-2.7, 8.882, -2.114]])
7  expected = np.array([[9.67704535e-01, 8.24424686e-04, 9.99917283e-01],
8                      [6.29733561e-02, 9.99861153e-01, 1.07743524e-01]])
```

For the first example, no broadcasting is needed because x is just a number.

For the second example, the things happen in a more interesting way. First, we see that `x` is an array, so NumPy applies the operator `-` to each element:

```
1  -x → [0, -1, -2]
```

Then, the exponential is applied element by element:

```
1  np.exp(-x) → [0e, -e1, -e2]
```

Now, we have `1` (a single number) and `np.exp(-x)` (an array). Broadcasting is automatically applied when NumPy treats the `1` as if it were copied to match the array:

```
1  1 → [1, 1, 1]
```

So the operation becomes:

```
1  [1, 1, 1] + [0e, -e1, -e2]
```

Although this addition is now done element by element, the `[1, 1, 1]` array is not really created — NumPy does this logically.

Finally, the division occurs when we divide `1` (a single number) and denominator (an array). Here, again, NumPy broadcasts the `1`:

```
1  1 → [1, 1, 1]
```

In such a way, the division is done element by element. Thus, you can see that **broadcasting** happens whenever a NumPy operation mixes a scalar (like `1`) with an array.

Performing operations along an axis

When working with NumPy arrays, especially two-dimensional arrays (matrices), we often want to apply an operation by rows or by columns instead of to the whole array at once. NumPy allows us to do this using the concept of an axis. In NumPy, **axis 0** refers to the rows direction (down the rows), while **axis 1** refers to the columns direction (across the columns).

NumPy also has other useful functions like `np.sum()`, `np.max()` and `np.min()`. When you just pass the Numpy array to these functions, they simply return the answer of the operation **over the entire array**.

Now suppose you want to find the maximum element of each column of an array. You can do this by passing an additional argument called `axis` to the function. For instance, if `A` is a two dimensional array, if you want to find the maximum of each column of `A`, you can use `np.max(A, axis=0)`. Conversely, if `A` is a two dimensional array, you use `np.max(A, axis=1)` if you want to find the maximum of each row of `A`.

```
1  # Minimum of all elements of array
2
3  a = np.array([[2, 1, 3],
4                [4, 5, 6]])
5
```

```
6 print(a)
7 print(np.min(a))
```

```
1 [[2 1 3]
2   [4 5 6]]
3 1
```

```
1 # Sum of all columns of an array, returned as one dimensional array
2 print(np.sum(a, axis=0))
```

```
1 [6 6 9]
```

A point to note is that these functions may return one dimensional arrays, which might cause errors with broadcasting. In order to make sure that two dimensional arrays are returned, you must pass the argument `keepdims=True` in the function: `np.max(a, axis=0, keepdims=True)`

```
1 # Sum of all columns of an array, returned as a two dimensional array
2 print(np.sum(a, axis=0, keepdims=True))
```

```
1 [[6 6 9]]
```

```
1 # Sum of all rows of an array
2 print(np.sum(a, axis=1, keepdims=True))
```

```
1 [[ 6]
2  [15]]
```

Exercise 3: Normalizing all columns of an array

Normalization means rescaling values so that they follow a common rule. A very common type of normalization is to divide each value in a column by a number related to that column (for example, the sum or the maximum of the column).

In many machine learning applications, **data** is normalized before training a model to improve performance and stability. This is because different columns of the data often represent different quantities and can be on very different scales. For example, one column might contain ages (values around tens), while another column might contain income (values in thousands). If we do not normalize the data, columns with larger numerical values can dominate the learning process, even if they are not more important. Normalization rescales each column so that all columns are treated more fairly by the model.

In practice, this means that **each column is handled separately**: we compute statistics (such as the mean and the standard deviation) for one column, and then use them to rescale the values in that same column. In this way, each column can be seen as a **vector of values** that is normalized independently from the others.

As a result, after normalization, all columns have a **similar scale**, which usually helps machine learning algorithms train faster and produce better results.

Suppose that:

- x_i is the i^{th} column of the input array
- c_i is the i^{th} column of the output (normalized) array

The normalization is defined as:

$$c_i = \frac{x_i - \text{mean}(x_i)}{\sigma(x_i)}$$

where:

- $\text{mean}(x_i)$ is the average (mean) of all the elements in column x_i
- $\sigma(x_i)$ is the standard deviation of all the elements in column x_i

You may revise `np.mean()` and `np.std()` for convenience.

For this exercise, you will **normalize all the columns** of a two-dimensional NumPy array. You may assume that $\sigma(x)$ is never equal to 0. Try to complete this exercise without using any for loops.

NumPy functions make use of a technique called vectorization which make them much faster than for loops. Using vectorized implementations can often make magnitudes of difference in the training time of your models: where your model might initially take a couple of days to train, it would now be done in a couple of hours.

```
1  import numpy as np
2
3  def normalize(x):
4      """
5      Normalize all the columns of x
6
7      Arguments:
8      x : numpy.ndarray
9          A two dimensional Numpy array
10
11     Returns:
12     c : numpy.ndarray
13         The normalized version of x
14     """
15
16     ### BEGIN SOLUTION
17
18     # Step 1. Calculate the mean of all columns
19     mean = None # Replace the None with the required expression for mean
20
21     # Setep 2. Calculate the standard deviation all columns
22     sigma = None # Replace the None with the required expression for sigma
23
24     # Step 3. Compute the final answer
```



```
25     c = (x - mean) / sigma
26
27     ### END SOLUTION
28
29     return c
```

Verify your solutions `c` for this exercise by computing the expected values for the following arrays.

```
1  x = np.array([[1, 4],
2                [3, 2]])
3  expected = np.array([[-1, 1],
4                       [1, -1]])
5
6
7  x = np.array([[324.33, 136.11, 239.38, 237.17],
8                [123.43, 325.24, 243.52, 745.25],
9                [856.36, 903.02, 430.25, 531.35]])
10 expected = np.array([[-0.35694152, -0.97689929, -0.73023324, -1.28391946],
11                      [-1.00662188, -0.39712973, -0.68372539,  1.1554411 ],
12                      [ 1.3635634 ,  1.37402902,  1.41395863,  0.12847837]])
```

1.8 Linear Algebra

Linear algebra is the branch of mathematics that works with **vectors** and **matrices**, and it is a fundamental tool in science, engineering, and machine learning. In Python, the NumPy library provides simple and efficient ways to create, manipulate, and compute with vectors and matrices. Using NumPy, we can perform operations such as matrix addition, multiplication, transposition, and solving systems of equations with just a few lines of code. Because NumPy uses optimized, vectorized implementations, these operations are not only easy to write but also very fast, making it an ideal tool for learning and applying linear algebra in practice.

In this notebook, we will represent vectors using either **one-dimensional NumPy arrays** or **two-dimensional column arrays** with shape $(d, 1)$. We will represent matrices using **two-dimensional Numpy arrays**.

```
1  v = np.arange(6)
2  print('Vector:\n', v)
3  print("Shape of vector:", v.shape)
4
5  M = np.array([[1, 2, 3],
6                [4, 5, 6]])
7  print('\nMatrix:\n', M)
8  print("Shape of matrix:", M.shape)
```

```
1  Vector:
2  [0 1 2 3 4 5]
3  Shape of vector: (6,)
4
5  Matrix:
6  [[1 2 3]
```

```
7 [4 5 6]]
8 Shape of matrix: (2, 3)
```

Transposing a Matrix

Transposing a matrix means **swapping its rows and columns**.

- The first row becomes the first column
- The second row becomes the second column
- And so on...

So, if a matrix has shape (m, n) , its transpose will have shape (n, m) . In NumPy, you can transpose a matrix using the `.T` attribute (or the `np.transpose()` function). For example, if `M` is a matrix, then you can compute its transpose using `M.T`.

```
1 M = np.array([[5, 2, 9],
2               [6, 1, 0]])
3
4 print('M\n', M)
5 print('\nTranspose of M (M.T)\n', M.T)
6
7
8 B = np.arange(9).reshape((3, 3))
9
10 print('\nB\n', B)
11 print('\nB.T', B.T)
```

```
1 M
2 [[5 2 9]
3  [6 1 0]]
4
5 Transpose of M (M.T)
6 [[5 6]
7  [2 1]
8  [9 0]]
9
10 B
11 [[0 1 2]
12  [3 4 5]
13  [6 7 8]]
14
15 B.T
16 [[0 3 6]
17  [1 4 7]
18  [2 5 8]]
```

Note that in NumPy, taking the transpose of a one-dimensional (1D) array has **NO** effect.

This is because a 1D array does not have rows and columns—it is just a list of numbers with shape $(n,)$. Since there is no distinction between rows and columns, NumPy cannot swap them, so the transpose looks exactly the same.

```
1 a = np.ones(3)
2 print('a:', a)
3 print('Shape of a:', a.shape)
4 print('\na.T:', a.T)
5 print('Shape of a.T:', a.T.shape)
```

```
1 a:
2 [1. 1. 1.]
3 Shape of a: (3,1)
4
5 A.T:
6 [1. 1. 1.]
7 Shape of a.T: (3,1)
```

However, if you use a 2D array with shape $(d, 1)$ (a column vector), then transposing does change its shape and structure.

In this case:

- The original array has shape $(d, 1)$ -> a column vector
- Its transpose will have shape $(1, d)$ -> a row vector

```
1 a = np.ones((3,1))
2 print('a:\n', a)
3 print('Shape of a:', a.shape)
4
5 print('\na.T:\n', a.T)
6 print('Shape of a.T:', a.T.shape)
```

```
1 a:
2 [[1.]
3  [1.]
4  [1.]]
5 Shape of a: (3, 1)
6
7 a.T:
8 [[1. 1. 1.]]
9 Shape of A.T: (1, 3)
```

Dot Product and Matrix Product

In linear algebra, we often need to **combine vectors and matrices using multiplication**. Two of the most common operations are the **dot product** and the **matrix product**.

- The dot product takes two vectors and produces a single number (a **scalar**).
- The matrix product combines matrices (or a matrix and a vector) to produce a **new vector or matrix**.

In NumPy, you can compute these using `np.dot(A, B)` - Or the `@` operator

You can revise the documentation of [np.dot\(\)](#) for more details.

```
1 def dot(a, b):
2     """Compute dot product between a and b.
3     Args:
4         a, b: (2,) ndarray as R^2 vectors
5
6     Returns:
7         a number which is the dot product between a, b
8     """
9
10    dot_product = a[0]*b[0] + a[1]*b[1]
11
12    return dot_product
13
14 # Test your code
15 a = np.array([1,0])
16 b = np.array([0,1])
17 print(dot(a,b)) # Should output 0
```

```
1 0.0
```

This makes geometric sense due to the vectors $[1,0]$ and $[0,1]$ are perpendicular (orthogonal), so their dot product is 0. That is:

- $a = [1, 0], b = [0, 1]$
- $a[0]$ and $a[1]$ access the first and second components of vector a
- $b[0]$ and $b[1]$ access the first and second components of vector b
- $a[0]*b[0] = 1 * 0 = 0$
- $a[1]*b[1] = 0 * 1 = 0$
- $\text{Sum} = 0 + 0 = 0$

Both `np.dot(A, B)` or the `@` operator do the same thing, but the `@` operator is shorter, cleaner, and closer to mathematical notation. So, from now on, we will use only the `@` operator.

```
1 x = np.array([1, 2, 3])
2 y = np.array([4, 5, 6])
3
4 print('x:', x)
5 print('y:', y)
6
7 print('\nnp.dot(x, y):', np.dot(x, y))
8 print('x @ y:', x @ y)
```

```
1 x: [1 2 3]
2 y: [4 5 6]
3
4 np.dot(x, y): 32
5 x @ y: 32
```

You can also use the `@` operator to compute the matrix product between two matrices or between a matrix and a vector (written as a column vector).

In both cases, the dimensions must be compatible: - For `A @ B`, the number of columns of `A` must match the number of rows of `B`. - For `A @ v`, the number of columns of `A` must match the size of the vector `v`.

```
1  A = np.array([[2, 0, 1],
2               [1, 3, 4],
3               [0, 2, 1]])
4
5  B = np.arange(6).reshape(3, 2)
6
7  C = np.array([3, 2, 8]).reshape((-1, 1))  # column vector
8
9  print('A:\n', A)
10 print('B:\n', B)
11 print('A @ B:\n', A @ B)
12
13 print('\nA:\n', A)
14 print('C:\n', C)
15 print('A @ C:\n', A @ C)
```

```
1  A:
2  [[2 0 1]
3   [1 3 4]
4   [0 2 1]]
5  B:
6  [[0 1]
7   [2 3]
8   [4 5]]
9  A @ B:
10 [[ 4  7]
11  [22 30]
12  [ 8 11]]
13
14 A:
15 [[2 0 1]
16  [1 3 4]
17  [0 2 1]]
18 C:
19 [[3]
20  [2]
21  [8]]
22 A @ C:
23 [[14]
24  [41]
25  [12]]
```

In the next example, we explore how different inner products change the way we measure angles between vectors. The standard Euclidean dot product gives us our usual geometric intuition, but many applications require modified inner products that weigh dimensions differently.

The code below demonstrates this concept by defining an inner product through a symmetric matrix **A**, which acts as a weighting matrix.

```

1  A = np.array([[1, -1/2],[-1/2,5]])      # the matrix A defines the inner product
2  x = np.array([0,-1])
3  y = np.array([1,1])
4
5  def find_angle(A, x, y):
6      """Compute the angle"""
7      inner_prod = x.T @ A @ y
8      norm_x = np.sqrt(x.T @ A @ x)      # length of x (norm_x) in this inner product
9      norm_y = np.sqrt(y.T @ A @ y)      # length of y (norm_y) in this inner product
10     alpha = inner_prod/(norm_x*norm_y)
11     angle = np.arccos(alpha)
12     return np.round(angle,2)
13
14 print(find_angle(A, x, y))

```

```

1  2.69

```

When using the @ operator for matrix multiplication, the shapes of the matrices must be compatible

Remember the rule: - If A has shape (m, n) - And B has shape (n, p)

Then A @ B is valid and will have shape (m, p).

If the number of columns of A does not match the number of rows of B, NumPy cannot perform the multiplication and will raise an error.

```

1  A = np.array([[1, 2, 3],
2                [4, 5, 6]])      # Shape: (2, 3)
3
4  B = np.array([[1, 2],
5                [3, 4]])        # Shape: (2, 2)
6
7  print("A shape:", A.shape)
8  print("B shape:", B.shape)
9
10 # This will cause an error because the shapes are not compatible
11 A @ B

```

```

1  -----
2
3  ValueError                                Traceback (most recent call last)
4
5  ValueError: matmul: Input operand 1 has a mismatch in its core dimension 0, with
   gufunc signature (n?,k),(k,m?)->(n?,m?) (size 3 is different from 2)

```

Exercise (Optional): Computing Angles with a Non-Standard Inner Product

For vectors **x** and **y** in $\mathbb{R}^2$, the inner product is defined by a symmetric positive definite matrix **A** is:

$$\langle \mathbf{x}, \mathbf{y} \rangle = \mathbf{x}^T A \mathbf{y}$$

The angle θ between vectors is then computed using the familiar cosine formula adapted to this new inner product:

$$\cos \theta = \frac{\langle \mathbf{x}, \mathbf{y} \rangle}{\sqrt{\langle \mathbf{x}, \mathbf{x} \rangle} \cdot \sqrt{\langle \mathbf{y}, \mathbf{y} \rangle}}$$

In this exercise, compute the angle between

$$\mathbf{x} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

and

$$\mathbf{y} = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

using the inner product defined by

$$\langle \mathbf{x}, \mathbf{y} \rangle = \mathbf{x}^T \begin{bmatrix} 1 & 0 \\ 0 & 5 \end{bmatrix} \mathbf{y}$$

```
1 import numpy as np
2
3 # Step 1. Fill in the arrays
4 A = np.array( )
5 x = np.array( )
6 y = np.array( )
7
8 # Step 2. Define the function `find_angle`
9 def find_angle(A, x, y):
10     pass
11     return np.round(angle, 4)
12
13 print(find_angle(A, x, y))
```

To verify your answer, use these arrays

```
1 A = np.array([[1, 0], [0, 5]])
2 x = np.array([1, 1])
3 y = np.array([1, -1])
4
```

5 2.3 rad

Lastly, remember that in NumPy, the operators `*` and `@` do not mean the same thing:

- `*` performs element-wise multiplication Each element is multiplied by the element in the same position.
- `@` performs matrix multiplication (linear algebra product) Rows of the first matrix are combined with columns of the second matrix.

This is a very common source of confusion for beginners, so it's important to keep the difference in mind.

```
1 A = np.array([[1, 2],
2               [3, 4]])
3
4 B = np.array([[5, 6],
5               [7, 8]])
6
7 print("A:\n", A)
8 print("B:\n", B)
9
10 print("\nA * B (element-wise multiplication):\n", A * B)
11 print("\nA @ B (matrix multiplication):\n", A @ B)
```

```
1 A:
2 [[1 2]
3  [3 4]]
4 B:
5 [[5 6]
6  [7 8]]
7
8 A * B (element-wise multiplication):
9 [[ 5 12]
10 [21 32]]
11
12 A @ B (matrix multiplication):
13 [[19 22]
14  [43 50]]
```

Outer product

The **outer product** of two vectors produces a matrix. The outer product of two vectors can be calculated using the function `np.outer()`. The function `np.outer()` creates a matrix where each row is a scaled version of `b`.

There is a key difference with **dot product**:

- ab^T = Outer product → Creates a **matrix** from two vectors
- $a^T b$ = Dot product → Computes a **scalar** from two vectors

```
1 a = np.array([1, 2, 3])      # length 3
2 b = np.array([4, 5])         # length 2
```



```
3
4 outer = np.outer(a, b)
5 print(outer)
```

```
1 [[ 4  5]
2   [ 8 10]
3   [12 15]]
```

```
1 u = np.array([1, 2, 2])
2 outer_uu = np.outer(u, u)
3 print(outer_uu)
```

```
1 [[1 2 2]
2   [2 4 4]
3   [2 4 4]]
```

This operation is important in projection matrices, where we need bb^T to create $D \times D$ matrix:

```
1 b = np.array([1, 2, 2])
2 bbT = np.outer(b, b)      # 3x3 matrix
3 bTb = np.dot(b, b)        # scalar 9
4
5 P = bbT / bTb              # projection matrix
6
7 print("bb^T (outer product):")
8 print(bbT)
9 print("\nb^Tb (dot product):")
10 print(bTb)
11 print("\nProjection matrix P = bb^T / b^Tb:")
12 print(P)
```

```
1 bb^T (outer product):
2 [[1 2 2]
3   [2 4 4]
4   [2 4 4]]
5
6 b^Tb (dot product):
7 9
8
9 Projection matrix P = bb^T / b^Tb:
10 [[0.11111111 0.22222222 0.22222222]
11   [0.22222222 0.44444444 0.44444444]
12   [0.22222222 0.44444444 0.44444444]]
```

The `numpy.linalg` library

NumPy provides a special module called `numpy.linalg` that contains many useful tools for linear algebra.

With `numpy.linalg`, you can easily:

- Solve systems of linear equations
- Compute determinants and inverses of matrices
- Compute eigenvalues and eigenvectors
- Compute matrix norms
- And perform many other common linear algebra operations

Instead of implementing these operations yourself, you can rely on this library, which is fast, well-tested, and easy to use. You can revise the documentation of [np.linalg](#) for more details.

For example, we can compute the **determinant** of a square matrix by using [np.linalg.det\(\)](#).

```
1 A = np.array([[2, 0, 1],
2               [1, 3, 4],
3               [0, 2, 1]])
4
5 detA = np.linalg.det(A)      # This computes the determinant
6 print("det(A) =", detA)
```

```
1 det(A) = 8.0
```

Note: Remember that small numerical differences (like 8.000000000000002) can sometimes appear because of floating-point arithmetic

We can compute the inverse of a matrix by using [np.linalg.inv\(\)](#).

The inverse of a matrix A is another matrix A^{-1} such that $A @ A^{-1} = I$, where I is the identity matrix.

```
1 A = np.array([[2, 0, 1],
2               [1, 3, 4],
3               [0, 2, 1]])
4
5 A_inv = np.linalg.inv(A)      # Compute the inverse of A
6
7 print("A:\n", A)
8 print("\nInverse of A:\n", A_inv)
9
10
11 I = np.eye(3)                # Generate the identity matrix
12 print("\nIdentity matrix I:\n", I)
13
14 # Check that A @ A_inv is (approximately) the identity matrix
15 print("\nA @ A_inv:\n", A @ A_inv)
```

```
1 A:
2 [[2 0 1]
3  [1 3 4]
4  [0 2 1]]
5
6 Inverse of A:
7 [[-0.625  0.25  0.875]
8  [ 0.125  0.25 -0.375]]
```

```
9  [ 0.25 -0.5  0.75 ]]
```

```
10
```

```
11 Identity matrix I:
```

```
12 [[1. 0. 0.]
```

```
13  [0. 1. 0.]
```

```
14  [0. 0. 1.]]
```

```
15
```

```
16 A @ A_inv:
```

```
17 [[1. 0. 0.]
```

```
18  [0. 1. 0.]
```

```
19  [0. 0. 1.]]
```

Note: Not every matrix has an inverse (the determinant must be non-zero). Because of floating-point arithmetic, results are usually approximately the identity matrix, not perfectly exact. In this last example, we use `np.eye(3)` to create a 3×3 identity matrix.

We can compute the eigenvalues and eigenvectors of a matrix using `np.linalg.eig()`.

This function returns two objects:

- An array with the eigenvalues
- A matrix whose columns are the corresponding eigenvectors

Let's use a simple symmetric matrix so the result is easy to interpret:

```
1  A = np.array([[2, 1],
```

```
2                [1, 2]])
```

```
3
```

```
4  # Compute the eigenvalues and eigenvectors
```

```
5  eigenvalues, eigenvectors = np.linalg.eig(A)
```

```
6
```

```
7  print("A:\n", A)
```

```
8  print("\nEigenvalues:\n", eigenvalues)
```

```
9  print("\nEigenvectors (columns):\n", eigenvectors)
```

```
1  A:
```

```
2  [[2 1]
```

```
3  [1 2]]
```

```
4
```

```
5  Eigenvalues:
```

```
6  [3. 1.]
```

```
7
```

```
8  Eigenvectors (columns):
```

```
9  [[ 0.70710678 -0.70710678]
```

```
10 [ 0.70710678  0.70710678]]
```

So far, we have seen how to compute eigenvalues and eigenvectors using `np.linalg.eig()`. But what do these results actually mean?

By definition, if v is an eigenvector of a matrix A with eigenvalue λ , then multiplying the matrix by this vector does not change its direction—it only scales it:

$$A@v = \lambda v$$

In the following example, we will take one of the eigenvalues and eigenvectors returned by NumPy and verify this property step by step using simple matrix and vector operations.

```
1  A = np.array([[2, 1],
2               [1, 2]])
3
4  # Compute eigenvalues and eigenvectors
5  eigenvalues, eigenvectors = np.linalg.eig(A)
6
7  print("A:\n", A)
8  print("\nEigenvalues:\n", eigenvalues)
9  print("\nEigenvectors (columns):\n", eigenvectors)
10
11 # Take the first eigenvalue and eigenvector
12 lambda_0 = eigenvalues[0]
13 v_0 = eigenvectors[:, 0]
14
15 print("\nFirst eigenvalue (lambda):", lambda_0)
16 print("First eigenvector (v):\n", v_0)
17
18 # Compute both sides of A @ v = lambda * v
19 left = A @ v_0
20 right = lambda_0 * v_0
21
22 print("\nA @ v:\n", left)
23 print("\nlambda * v:\n", right)
```

```
1  A:
2  [[2 1]
3   [1 2]]
4
5  Eigenvalues:
6  [3. 1.]
7
8  Eigenvectors (columns):
9  [[ 0.70710678 -0.70710678]
10   [ 0.70710678  0.70710678]]
11
12 First eigenvalue (lambda): 3.0
13 First eigenvector (v):
14 [0.70710678 0.70710678]
15
16 A @ v:
17 [2.12132034 2.12132034]
18
19 lambda * v:
20 [2.12132034 2.12132034]
```

Note: An **eigenvector** keeps its direction when multiplied by the matrix, while the **eigenvalue** tells you how much it is stretched or shrunk

Sorting eigenvalues and reordering eigenvectors

Suppose that we want the eigenvalues in the diagonal matrix D to appear in **non-decreasing order**. However, `np.linalg.eig(A)` does **not** guarantee any particular order for the eigenvalues. Therefore, we need to **sort them manually**.

```
1 eig_vals = np.array([6, -1])
```

The function `np.argsort(eig_vals)` does **not** sort the values directly. Instead, it returns the **indices** that would sort the array.

```
1 eig_vals = np.array([6, -1])
2 idx = np.argsort(eig_vals)
3 print(idx)
```

```
1 [1 0]
```

This means:

- The smallest value is at index 1 (which is -1)
- The next value is at index 0 (which is 6)

So `idx` tells us the order of indices that sorts `eig_vals`.

NumPy also allows indexing an array using another array of indices. This is often called **fancy indexing**.

```
1 eig_vals = np.array([6, -1])
2 idx = np.array([1, 0])
3
4 eig_vals_sorted = eig_vals[idx]
5 print(eig_vals_sorted)
```

```
1 [-1  6]
```

So the line `eig_vals = eig_vals[idx]` means “reorder `eig_vals` using the index order stored in `idx`.”

Recall that S is a matrix whose **columns are eigenvectors**, and each column corresponds to one eigenvalue in `eig_vals`.

If we reorder the eigenvalues, we must also reorder the columns of S in the same way, so that each eigenvector still matches its eigenvalue.

The syntax:

```
1 S[:, idx]
```

means:

- `:` $\rightarrow$ take all rows
- `idx` $\rightarrow$ take only the columns in the order given by `idx`

So the line `S = S[:, idx]` means “reorder the columns of `S` using the same index order used to sort the eigenvalues.”

We can write down all the lines together,

```
1 idx = np.argsort(eig_vals)
2 eig_vals = eig_vals[idx]
3 S = S[:, idx]
```

that indicates:

- Compute the index order that sorts the eigenvalues
- Reorder the eigenvalues using that order
- Reorder the columns of `S` in the same way, so each eigenvector still corresponds to its eigenvalue

Note: Remember that the function `np.linalg.eig()` does not guarantee sorted eigenvalues.

When we say that eigenvalues should be in **non-decreasing order**, we mean they should be ordered from **smallest to largest** (allowing equal values if any). For example, suppose a matrix has the following eigenvalues (as returned by NumPy, in arbitrary order):

```
1 eig_vals = np.array([6, -1, 3, 3])
2 print(eig_vals)
```

```
1 [ 6 -1  3  3 ]
```

These eigenvalues are not sorted. To **sort** them in **non-decreasing order**, we can do:

```
1 idx = np.argsort(eig_vals)
2 eig_vals_sorted = eig_vals[idx]
3
4 print("Sorted eigenvalues:", eig_vals_sorted)
```

```
1 Sorted eigenvalues: [-1  3  3  6]
```

Exercise 4: Diagonalizing a Matrix

Diagonalization is an important concept in linear algebra that builds directly on **eigenvalues** and **eigenvectors**.

A square matrix A is said to be **diagonalizable** if it can be written in the form $A = SDS^{-1}$,

where:

- S is a matrix whose columns are the eigenvectors of A ,
- D is a diagonal matrix containing the corresponding eigenvalues of A ,
- S^{-1} is the inverse of S .

You might find `np.zeros()`, `np.linalg.eig()` and `np.linalg.inv()` useful. Note that for this exercise, you may assume that A is always diagonalizable.

For testing purposes:

- Each eigenvector in S must be of **unit length**. This is automatically satisfied if you use `np.linalg.eig()`. If you do not, you may need to normalize the eigenvectors yourself.
- The eigenvalues in D must appear in **non-decreasing order**.

The idea behind diagonalization is to rewrite a matrix in a simpler form (diagonal), which makes many computations easier, such as computing powers of a matrix or understanding how the matrix acts on vectors.

In this exercise, you need to find matrices S and D . Recall that in order to do this, you must first find all the eigenvalues and eigenvectors of A . Then, S is the matrix of all the eigenvectors arranged as columns, and D is the matrix of the corresponding eigenvalues arranged along the diagonal.

Note: Eigenvectors are not unique. If v is an eigenvector, then any non-zero multiple of v is also a valid eigenvector. For this reason, there are many possible correct matrices S . Different implementations (or NumPy itself) may return eigenvectors that differ by a scaling factor or by a sign. As long as the columns of S are valid eigenvectors corresponding to the eigenvalues in D , the diagonalization is correct. In this exercise, we will use normalized (unit-length) eigenvectors, as returned by `np.linalg.eig()`.

Suppose

$$A = \begin{bmatrix} 1 & 5 \\ 2 & 4 \end{bmatrix}$$

Then, one possible choice is

$$S = \begin{bmatrix} -2.5 & 1 \\ 1 & 1 \end{bmatrix}$$

and

$$D = \begin{bmatrix} -1 & 0 \\ 0 & 6 \end{bmatrix}$$

Note that this is just one valid diagonalization. NumPy will typically return normalized eigenvectors, so the matrix S you obtain in practice may look different, even though it represents the same eigen-directions and still satisfies $A = SDS^{-1}$.

```
1 import numpy as np
2
3 def diagonalize(A):
4     """
5     Diagonalizes the input matrix A.
6
```

```

7     Parameters:
8     A : np.ndarray
9         A two-dimensional NumPy array which is guaranteed to be diagonalizable.
10
11     Returns:
12     S : np.ndarray
13         Matrix whose columns are the eigenvectors of A.
14     D : np.ndarray
15         Diagonal matrix of eigenvalues.
16     S_inv : np.ndarray
17         Inverse of S.
18     """
19
20     ### BEGIN SOLUTION
21
22     # Step 1. Retrieve the number of rows in A
23     n = 0
24
25     # Step 2. Get the eigenvalues and eigenvectors of A
26     eig_vals, S = None, None
27
28     # Step 3. Start by initializing D to a matrix of zeros of the appropriate shape
29     D = None
30
31     # Step 4. Set the diagonal elements of D to be the eigenvalues
32     for i in range(n):
33         pass
34
35     # Step 5. Compute the inverse of S
36     S_inv = None
37
38     ### END SOLUTION
39
40     return S, D, S_inv

```

Verify your solutions S , D , S_{inv} for this exercise by computing the expected values for the following (2x2) and (4x4) arrays.

```

1  A = np.array([[1, 5],
2                [2, 4]])
3
4  S, D, S_inv = diagonalize(A)
5
6  print("A:\n", A)
7  print("\nS:\n", S)
8  print("\nD:\n", D)
9  print("\nS_inv:\n", S_inv)
10
11 print("\nReconstructed A (S @ D @ S_inv):\n", S @ D @ S_inv)
12 print("\nAll close?", np.allclose(A, S @ D @ S_inv))

```



```
1 A = np.array([[4, -9, 6, 12],
2               [0, -1, 4, 6],
3               [2, -11, 8, 16],
4               [-1, 3, 0, -1]])
5
6 S, D, S_inv = diagonalize(A)
7
8 print("A:\n", A)
9 print("\nS:\n", S)
10 print("\nD:\n", D)
11 print("\nS_inv:\n", S_inv)
12
13 # Reconstruct A using the diagonalization
14 A_reconstructed = S @ D @ S_inv
15
16 print("\nReconstructed A (S @ D @ S_inv):\n", A_reconstructed)
17
18 # Check if the reconstruction is correct (up to numerical precision)
19 print("\nIs the reconstruction correct? ", np.allclose(A, A_reconstructed))
```

Exercise 5: Polynomial Multiplication

A **polynomial** is an expression made of variables and coefficients, such as:

$$p(x) = 2x^2 + 3x + 1$$

and

$$q(x) = x + 4$$

Multiplying two polynomials means **distributing** every term of the first polynomial over every term of the second polynomial, and then **combining like terms**.

For example:

$$\begin{aligned}(2x^2 + 3x + 1)(x + 4) \\&= 2x^3 + 8x^2 + 3x^2 + 12x + x + 4 \\&= 2x^3 + 11x^2 + 13x + 4\end{aligned}$$

So the product is another polynomial whose degree is the **sum of the degrees** of the original polynomials.

But, why is this related to arrays and linear algebra? If we store the coefficients of a polynomial in an array, polynomial multiplication becomes a **systematic combination of coefficients**, very similar to:

- Convolutions
- Dot products
- Matrix-vector products

For example, the polynomial:

$$p(x) = 2x^2 + 3x + 1$$

can be represented by the coefficient array:

```
1 [2, 3, 1]
```

And,

$$q(x) = x + 4$$

can be represented as:

```
1 [1, 4]
```

Multiplying these two arrays in the right way produces the coefficients of the product polynomial.

In this exercise, you will implement a function to multiply two polynomials using NumPy. This task brings together several concepts you have learned so far, such as **array slicing**, **dot products**, and **vectorization**.

You are given two **one-dimensional NumPy arrays** A and B containing the coefficients of two polynomials. We will use the convention that:

- $A[i] = a_i$ is the coefficient of (x^i) in the first polynomial
- $B[i] = b_i$ is the coefficient of (x^i) in the second polynomial

Your goal is to compute the coefficients of the product polynomial $C = A \cdot B$.

More formally, if C is the resulting one-dimensional array and $C[i] = c_i$, then:

$$c_i = \sum_{j+k=i} a_j b_k$$

There are multiple ways to implement polynomial multiplication. If your approach requires a NumPy function that we have not introduced yet, we encourage you to consult the [NumPy documentation](#). However, try to implement the function using only **one for loop over (i)**, and compute the summation using **NumPy operations** (instead of nested loops). This will make your code faster thanks to **vectorization**.

Hints: - A and B may have different lengths. Pad the **end** of the shorter array with zeros so they have the same length. You may find `np.pad()` useful. - For a fixed (i), notice how the valid indices (j) and (k) must satisfy $(j+k=i)$. This often leads to multiplying two **slices** of the arrays. The summation may resemble a **dot**

product. - You can reverse an array using `np.flip()`, which can be helpful for aligning terms. - Make sure your output does **not** end with unnecessary trailing zeros. Look for a NumPy-based way to remove them.

```
1  import numpy as np
2
3  def multiply(A, B):
4      """
5      Multiplies two polynomials represented by their coefficient arrays.
6
7      Parameters
8      -----
9      A : np.ndarray
10         Coefficients of the first polynomial.
11      B : np.ndarray
12         Coefficients of the second polynomial.
13
14      Returns
15      -----
16      C : np.ndarray
17         Coefficients of the product polynomial A * B.
18      """
19
20     ### BEGIN SOLUTION
21
22     # Step 1. Find the number of coefficients of both polynomials
23     na = None
24     nb = None
25
26     # Step 2. Pad the smaller array with zeros so A and B have the same length
27     if False:
28         pass
29     else:
30         pass
31
32     # Step 3. Initialize the output array with zeros
33     C = None
34
35     # Step 4. Perform the multiplication
36     # You might want to break the loop over i into two separate phases
37     pass
38
39     # Step 5. Remove any extra zeros from the end of C
40     pass
41
42     ### END SOLUTION
43
44     return C
```

Verify your solutions C for this exercise by computing the expected values for the following arrays.

```
1  # Test case 1
2  A = np.array([1, 2])
```

```

3 B = np.array([3, 4])
4 C_exp = np.array([3, 10, 8])
5
6 C = multiply(A, B)
7 print("Test 1 result: ", C)
8 print("Test 1 expected: ", C_exp)
9 print("Test 1 correct? ", np.allclose(C, C_exp))
10
11 print()
12
13
14 #Test case 2
15 A = np.array([5, 6])
16 B = np.array([1, 3, 5, 9])
17 C_exp = np.array([5, 21, 43, 75, 54])
18
19 C = multiply(A, B)
20 print("Test 2 result: ", C)
21 print("Test 2 expected: ", C_exp)
22 print("Test 2 correct? ", np.allclose(C, C_exp))

```

Exercise 6: Support Vector Classifiers

In machine learning methods, support vector machines (SVM) are algorithms or models used in applications of regression and classification. In this framework, the **support vectors** are the critical data points that shape the model. They are literally data **vectors** (points in feature space) that happen to define the SVM model, and constitute a **subset of your training rows** that the model actually uses to build the decision boundary. For example, if you draw a line to separate two groups, the points closest to that line are the support vectors. If you move them, the line moves. If you remove other points far away, the line usually stays the same.

After training an SVM, the model predicts using a formula of this form:

$$f(x) = \sum_{i \in SV} \alpha_i y_i K(x_i, x) + b$$

where:

- $f(x)$ is a real-valued function: $\mathbb{R}^n \rightarrow \mathbb{R}$.
- x = the new point (sample) you want to predict
- $x_i = (x_1, x_2, \dots, x_n)$ = training points, with n the number of features per i th training sample
- y_i = their labels for classification (e.g., +1, -1)
- $K(\cdot, \cdot)$ = kernel function (or dot product in linear SVM)
- α_i = learned weights
- b = bias
- SV = only the support vectors

The function $f(x)$ takes an input vector $x \in \mathbb{R}^n$ and outputs a real number $f(x) \in \mathbb{R}$.

In binary classification:

$$\begin{cases} f(x) > 0 & \Rightarrow \text{class } +1 \\ f(x) < 0 & \Rightarrow \text{class } -1 \\ f(x) = 0 & \Rightarrow \text{on the decision boundary} \end{cases}$$

The sign of $f(x)$ gives the class, while the magnitude $|f(x)|$ relates to how far you are from the boundary (in model space).

In the case of SVM used for classification (i.e. SVC), only support vectors appear in such a the prediction formula. Because of how SVM is trained (optimization with constraints), Each training point gets a coefficient α_i , then most $\alpha_i = 0$ and those points different to zero are the suport vectors. Mathematically, only support vectors survive in the final formula.

In this exercise, you will work with the **linear case**. You build a toy SVC decision function and evaluate it on a point. You will use 3D vectors, a linear kernel (i.e. a dot product), and a data set

$$X = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$$

where:

- Each $x_i \in \mathbb{R}^n$ is a vector (one sample). Here, $x_1, x_2, \dots, x_N$ are different elements (for the set X) with n number of features
- Each $y_i \in \{+1, -1\}$ is its label

From these, SVM learns the coefficients α_i , selects some of the x_i as support vectors and builds the decision function.

Assume after training, SVC found three support vectors in $\mathbb{R}^3$:

$$x_1 = (1, 0, 0), \quad y_1 = +1, \quad \alpha_1 = 1$$

$$x_2 = (0, 1, 0), \quad y_2 = -1, \quad \alpha_2 = 1$$

$$x_3 = (0, 0, 1), \quad y_3 = +1, \quad \alpha_3 = 0.5$$

Also, assume $b = 0$ and the kernel linear $K(x_i, x) = x_i \cdot x$

The decision formula is:

$$f(x) = 1(+1)(x_1 \cdot x) + 1(-1)(x_2 \cdot x) + 0.5(+1)(x_3 \cdot x)$$

Suposse a point for classification to be

$$x = (2, 1, 4)$$

So, after the dot product operations, the decision formula becomes

$$f(x) = 2 - 1 + 0.5(4) = 3$$

This implies that $f(x) > 0$, thus the point is predicted to be classified as class +1. In practice, you will have more a set of points (real sample) to be classified.

Write a **NumPy-only** script that reproduces the toy example: build the linear kernel matrix, identify support vectors from α , compute the decision formula $f(x)$, and the predicted class from the sign of $f(x)$.

```

1  import numpy as np
2
3  def svm_linear_predict(X, y, alpha, b, x_new):
4      """
5          Predicts the class for a new sample using a linear kernel.
6
7          Parameters
8          -----
9          X : np.ndarray, shape (N, D)
10             Training data with N samples and D features.
11          y : np.ndarray, shape (N,)
12             Training labels (+1 or -1 for each sample).
13          alpha : np.ndarray, shape (N,)
14             Learned alpha parameters from SVM training.
15          b : float
16             Learned bias term.
17          x_new : np.ndarray, shape (D,)
18             New sample to classify.
19
20          Returns
21          -----
22          f : float
23             Decision function value f(x).
24          y_pred : int
25             Predicted class (+1 or -1).
26          """
27
28      ### BEGIN SOLUTION
29
30      # Step 1: Find support vectors (indices where alpha != 0)
31      sv_idx = None
32
33      # Step 2: Compute kernel values between each training sample and x_new
34      # For linear kernel: k_i = X[i] dot x_new
35      k = None
36
37      # Step 3: Compute decision function f(x)
38      # f(x) = sum_{i in SV} alpha_i * y_i * K(x_i, x) + b

```

```
39     f = None
40
41     # Step 4: Predict class based on sign of f(x)
42     y_pred = None
43
44     ### END SOLUTION
45
46     return f, y_pred
47
48
49 def svm_linear_kernel_matrix(X):
50     """
51     Computes the linear kernel matrix for training data.
52
53     Parameters
54     -----
55     X : np.ndarray, shape (N, D)
56         Training data with N samples and D features.
57
58     Returns
59     -----
60     K : np.ndarray, shape (N, N)
61         Kernel matrix where  $K[i, j] = X[i] \cdot X[j]$ .
62     """
63
64     ### BEGIN SOLUTION
65
66     # Compute the kernel matrix using linear kernel (dot product)
67     # Hint: Use matrix multiplication
68     K = None
69
70     ### END SOLUTION
71
72     return K
```

Verify your solutions for this exercise by computing the expected values for the following arrays.

```
1  # Test case 1
2  X = np.array([
3      [1, 0, 2],
4      [0, 1, 1],
5      [2, 1, 0],
6      [1, 2, 1],
7  ], dtype=float)
8
9  y = np.array([+1, +1, -1, -1], dtype=float)
10 alpha = np.array([0.5, 0.0, 0.5, 0.0], dtype=float)
11 b = -0.2
12 x_new = np.array([1, 1, 1], dtype=float)
13
14
15 # Compute kernel matrix
```

```
16 K = svm_linear_kernel_matrix(X)
17 print("Kernel matrix K (linear):")
18 print(K)
19
20 # Make prediction
21 f, y_pred = svm_linear_predict(X, y, alpha, b, x_new)
22 print(f"\nDecision value f(x): {f}")
23 print(f"Predicted class: {y_pred}")
```

1.9 Saving and Loading Data with .npy and .npz Files

NumPy's .npy files are the **native binary format** for storing a single NumPy arrays. They are designed to save and load arrays efficiently and without losing any information.

You can think of an .npy file as an exact snapshot of an array. It stores not only the data, but also the shape, the data type (dtype), and the memory layout. This is especially useful when working with large datasets or expensive computations that you do not want to repeat every time you run your program.

However, you still might wonder why not use text formats like .txt or .csv. This is because the .npy format has three important advantages:

- Speed: Saving and loading are much faster because the data is stored in binary form, without needing to parse text.
- Size: Files are usually smaller thanks to efficient binary storage.
- Fidelity: All NumPy-specific information (such as dtype='complex128' or dtype='<U10' for strings) is preserved exactly.

NumPy provides two main functions for working with this format: np.save() and np.load(). To **save** an array, you provide a filename and the array. NumPy automatically adds the .npy extension. To **load** the array, NumPy reads the file and returns the array exactly as it was saved, ready to use.

```
1 data_array = np.array([[1, 2, 3], [4, 5, 6]], dtype=np.float64)
2
3 np.save('my_data.npy', data_array)      # save the array to disk
4
5 loaded_array = np.load('my_data.npy')    # load the array back
6
7 print(loaded_array)
8 print(f"\nType: {loaded_array.dtype}")
9 print(f"\nShape: {loaded_array.shape}")
```

```
1 [[1. 2. 3.]
2  [4. 5. 6.]]
3
4 Type: float64
5
6 Shape: (2, 3)
```

Let's consider a simple example. Suppose you have a raw **CSV** file raw_grades.csv that contains student grades, but with some inconsistencies:


```
1 student_id,math,physics,chemistry
2 101,85.5,92.0,78.5
3 102,90.0,88.5,NA
4 103,76.0,82.5,91.0
5 104,NA,95.0,87.5
6 105,88.0,91.5,84.0
```

Notice that some values are missing and are represented by the string "NA". Before using this data for analysis (or machine learning), these values must be **cleaned and handled properly**.

A typical workflow is:

- Load the raw data from the CSV file
- Convert the data to a NumPy array, treating "NA" as missing values
- Clean or replace the missing values (for example, with `np.nan` or with column averages)
- Save the cleaned array to a binary NumPy file (.npy) so it can be quickly reloaded later without repeating the cleaning step

For example, you can load the CSV and handle missing values like this:

```
1 # Load the data, telling NumPy that "NA" represents missing values
2 raw_data = np.genfromtxt(
3     "raw_grades.csv",
4     delimiter=",",
5     skip_header=1,
6     dtype=float,
7     missing_values="NA",
8     filling_values=np.nan
9 )
10
11 print("Raw data loaded into NumPy:")
12 print(raw_data)
13 print(f"\nShape: {raw_data.shape}")
14 print(f"\nHas NaN values: {np.any(np.isnan(raw_data))}")
```

```
1 Raw data loaded into NumPy:
2 [[101.  85.5  92.  78.5]
3  [102.  90.   88.5 nan]
4  [103.  76.   82.5 91. ]
5  [104.  nan  95.   87.5]
6  [105.  88.   91.5 84. ]]
7
8 Shape: (5, 4)
9
10 Has NaN values: True
```

At this point, the missing values are represented as `np.nan`, which NumPy understands and can handle in computations.

After cleaning or processing the data (for example, replacing `np.nan` with column means), you can save the result:

```
1 np.save("clean_grades.npy", raw_data)
```

Later, you can load the cleaned data instantly:

```
1 clean_data = np.load("clean_grades.npy")
2 print("Clean data loaded from .npy file:")
3 print(clean_data)
```

```
1 Clean data loaded from .npy file:
2 [[101.  85.5  92.   78.5]
3  [102.  90.   88.5   nan]
4  [103.  76.   82.5  91.  ]
5  [104.   nan  95.   87.5]
6  [105.  88.   91.5  84.  ]]
```

Note: It is a widely used convention and best practice in data analysis and machine learning that **rows** represent individual samples or subjects (e.g., students), while **columns** represent features or properties (e.g., math, physics, chemistry). This layout is used by NumPy, pandas, scikit-learn, and most ML libraries because it makes operations like statistics, filtering, and modeling consistent and convenient.

While .npy files are used to store a single NumPy array, the .npz format is used to store **multiple arrays in one file**.

You can think of an .npz file as a container that holds several .npy arrays together. Each array is stored with a name, so you can load them later and access each one separately. This is useful when you want to save related data together, such as:

- input data and labels
- multiple intermediate results
- or several arrays that belong to the same experiment

To save multiple arrays, you use `np.savez()`. To load them, you use `np.load()`, which returns an object that works like a dictionary, where each array can be accessed by its name.

PART 2. Introduction to Assert Statements and Testing

When you write code, it is not enough for it to **run without errors**—you also want to be sure that it produces the **correct results**. This is where **testing** becomes important.

In Python, one simple and very useful way to test your code is by using **assert statements**. An **assert** statement checks whether a condition is True. If it is, the program continues normally. If it is not, Python raises an error and tells you that something went wrong.

In other words, **assert** is a way of saying:

“I expect this to be true. If it is not, stop the program and tell me.”

This is extremely helpful when:

- You want to **verify** that your functions return the correct results
- You want to **catch mistakes early**
- You want to **debug** your code more easily

```
1 def add(a, b):
2     return a + b
3
4 # This should be true, so nothing happens
5 assert add(2, 3) == 5
6
7 # This is false, so Python will raise an error
8 assert add(2, 2) == 5
```

As showed, the `assert` statement lets you check if a condition in your program evaluates to true. If the condition does evaluate to true, then nothing happens and the program execution continues normally. However, if it evaluates to false, then the program execution is immediately terminated, an error is raised and an error message is printed.

Exercise 7: Using assert for Debugging

```
1 # An assert statement where condition evaluates to true
2 x = 5
3 assert x == 5
```

Since the condition evaluates to true, nothing happens.

```
1 # An assert statement where the condition evaluates to false
2 x = 5
3 assert x == 4
```

```
1 -----
2
3 AssertionError                                Traceback (most recent call last)
4
5     1 # An assert statement where the condition evaluates to false
6     2 x = 5
7 ----> 3 assert x == 4
8
9
10 AssertionError:
```

This time, since the assert statement evaluates to false, an error is thrown and the line where the assert statement failed is printed to the standard output. You can also print a message which further explains what error took place.

```
1 # An asseert statement with an error message
2 x = 5
```

```
3  assert x == 4, "x does not store the intended value"
```

```
1  -----
2
3  AssertionError                                Traceback (most recent call last)
4
5      1 # An asseert statement with an error message
6      2 x = 5
7  ----> 3 assert x == 4, "x does not store the intended value"
8
9
10 AssertionError: x does not store the intended value
```

In this exercise, you will use **assert statements** inside the functions you write to help you **debug** your code.

The **assert** statements are a very powerful tool for finding bugs. They allow you to **check that something you expect to be true is actually true**. If it is not, Python stops the program and shows an error, telling you exactly where the problem occurred.

Let's look at a simple example where an **assert** statement helps us **spot a bug**. Your goal is to **find the bug in the code and fix it**.

```
1  import numpy as np
2
3  def test_assert():
4      """
5      This function demonstrates the use of assert statements in debugging
6      """
7
8      A = np.arange(5)
9      s = 0
10
11     # Step 1, add all elements of A to s
12     for i in range(A.shape[0]):
13         s += A[i]
14
15     # Step 2, subtract all the elements of A in reverse order
16     for i in range(A.shape[0] - 1, -1, -1): # Unfortunately, there is a bug in
this loop
17         s -= A[i]
18
19     # If everything were correct, s should be 0 at this point
20     # This assert checks that assumption
21     assert s == 0
22
23 test_assert()
```

When you run this code, the assert statement will fail and raise an **AssertionError**, telling you that something went wrong. This means there is a **bug in the code** above.

Your task is to inspect the loops, find the mistake, and fix it so that the assertion passes.

Exercise 8: The Numpy Testing Module

NumPy provides a very useful testing utility called `np.testing.assert_allclose()` which allows us to test our functions.

We strongly encourage you to read the documentation of this function, as it is widely used in scientific and numerical computing.

Why not use `==` with floating-point numbers?

- When working with **floating-point numbers**, you often get **small rounding errors** due to the way numbers are represented in the computer. Because of this, two values that should be equal mathematically may differ by a very small amount
- For example, instead of getting exactly `1.0`, you might get something like `0.999999998` or `1.000000001`. In such cases, a direct comparison using `==` may incorrectly report that the numbers are different
- `np.testing.assert_allclose()` solves this problem by checking whether two values (or arrays) are **close enough**, rather than exactly equal

How does `np.testing.assert_allclose()` work? The function takes **two required arguments**:

- The value (or array) you want to test
- The expected value (or array) you want to compare against

It then checks whether they are **equal within a small tolerance**. If they are, nothing happens. If they are not, Python raises an error and tells you that the test failed.

You can also **customize the tolerance** if needed, depending on how strict you want the comparison to be.

This makes `np.testing.assert_allclose()` especially useful for:

- Testing functions that return **floating-point arrays**
- Verifying **numerical algorithms**
- Writing **reliable tests** for scientific code

In the next exercise, you are given a function that **attempts** to compute the inverse of a 2×2 matrix. However, this function is **incorrect**.

```
1 def inverse(A):
2     """
3     Computes (incorrectly) the inverse of A
4
5     A must have shape (2, 2)
6     """
7
8     return np.array([[A[1, 1], -A[0, 1]],
9                     [-A[1, 0], A[0, 0]]])
```

A test has already been written for this function, but unfortunately, that **test passes even though the function is wrong**. This means the test is not strong enough.

Your tasks are:

- Write a new test for this function that fails, showing that the current implementation is incorrect
- Fix the function so that it becomes correct
- Verify that your corrected function now passes the test you wrote

Here is one test case you can start with:

```
1 A = np.array([[3, 5],
2               [1, 2]])
3 A_exp = np.array([[2, -5],
4                  [-1, 3]])
```

Try to think of **another test case** where the current implementation should fail, and add it below:

```
1 A = np.array([
2
3     # Add your own test matrix here
4
5 ])
```

PART 3. Debugging Your Code

As you work through the rest of the sections in this notebook, you will inevitably encounter situations where your code does **not** work as expected. This is completely normal—and it happens to everyone, even experienced programmers.

Debugging can sometimes feel difficult or frustrating, especially when you do not immediately understand what went wrong. In this final part of the notebook, we will introduce some **practical guidelines and strategies** to help you find and fix bugs in your code more effectively.

The goal is not only to solve the current exercises, but also to help you become more **confident and systematic** when debugging your own programs in the future.

3.1 Errors in Matrix Multiplication

One of the most common mistakes when working with NumPy and linear algebra is trying to multiply arrays whose **dimensions are not compatible**.

Let's first look at what happens when we try to multiply two matrices with incompatible shapes:

```
1 A = np.zeros((3, 2))
2 B = np.zeros((4, 5))
3 print(A @ B)
```

```
1 -----
2
3 ValueError                                Traceback (most recent call last)
4
5 <ipython-input-86-c1fcd344e478> in <module>
6     1 A = np.zeros((3, 2))
7     2 B = np.zeros((4, 5))
8 ----> 3 print(A @ B)
9
10
11 ValueError: matmul: Input operand 1 has a mismatch in its core dimension 0, with
    gufunc signature (n?,k),(k,m?)->(n?,m?) (size 4 is different from 2)
```

That error message looks complicated, but for our purposes we only need to focus on the key part:

```
1 size 4 is different from 2
```

This tells us that:

- A has shape (3, 2) -> it has **2 columns**
- B has shape (4, 5) -> it has **4 rows**

For matrix multiplication $A @ B$ to work, the number of columns of A must match the number of rows of B.

Here, $2 \neq 4$, so the multiplication is not defined, and NumPy raises an error.

A simple and effective strategy is to print the shapes of your arrays before and after each matrix multiplication and check that they are compatible:

```
1 print(A.shape)
2 print(B.shape)
```

Exercise 9: Practicing Code Debugging

We would advise you to use `np.outer()` and `np.inner()` when computing the dot product of 1D arrays. If X is a vector (represented as a 1D array in this course), then `np.inner(X, X)` calculates $X^T \cdot X$ (the regular dot product) and `np.outer(X, X)` computes $X \cdot X^T$.

If you are performing matrix multiplication between a 2D array and a 1D array, we would advise you to first reshape the 1D array into a 2D array of shape $(d, 1)$.

In this exercise, you will practice **debugging NumPy code** and using **tests** to verify that your implementation is correct. The goal is not only to fix the code, but also to **understand why it was wrong** and how to reason about array shapes and operations.

Before starting, keep in mind the following useful NumPy functions:

- `np.inner()`: computes the dot product of two 1D arrays.
If X is a vector (represented as a 1D array in this course), then `np.inner(X, X)` computes $(X^T X)$, the usual dot product.

- `np.outer()`: computes the outer product.
For a vector X , `np.outer(X, X)` computes $(X X^T)$.
- When multiplying a **2D array** by a **1D array**, it is often helpful to first reshape the 1D array into a **column vector** with shape $(d, 1)$.

The function below is intended to compute the **sum of the maximum element of each row** of a 2D NumPy array. However, there is a **bug** in the implementation.

Your task is to **find and fix the bug** so that the function passes the provided tests.

```
1 import numpy as np
2
3 def sum_of_max(A):
4     """
5     Computes the sum of the maximum element of each row of A.
6
7     A must be a 2D NumPy array.
8     """
9     return np.sum(np.max(A, axis=1))
```

The following tests describe the expected behavior of the function. After you fix the implementation, both tests should pass.

```
1 A = np.array([[1, 2],
2               [3, 4]])
3 np.testing.assert_allclose(sum_of_max(A), 6)
4
5 A = np.array([[24, 69, 83],
6               [74, 14, 27]])
7 np.testing.assert_allclose(sum_of_max(A), 157)
```

Wrap-Up and Next Steps

Congratulations on making it to the end of this notebook! This was a long and demanding journey, and completing it is a great achievement.

By now, you should feel more comfortable working with **NumPy arrays**, performing **basic linear algebra operations**, and writing **cleaner, more reliable code** using **tests and assertions**. You have also practiced **debugging strategies** that will help you identify and fix problems more systematically in your future projects.

Remember that learning programming and numerical computing is a **gradual process**. You do not need to memorize everything—what matters most is that you now know:

- How to explore and use the documentation
- How to test your code
- How to debug problems when things go wrong

As a next step, you are encouraged to:

- Revisit the exercises and try to solve them again without looking at your previous solutions
- Modify the examples and see how the behavior changes
- Apply these tools to your own projects or to more advanced topics such as data analysis, machine learning, or scientific computing

Keep experimenting, keep breaking things (on purpose!), and keep learning. That's how you really become confident with NumPy and Python.

License

The content of this tutorial itself is licensed under the terms and conditions of the [Creative Commons Attribution \(CC BY 4.0\) license](#), and the underlying source code used to format and display that content is licensed under the [MIT license](#). See the LICENSE files for full details.

Attribution

If you use or adapt this material, please provide appropriate credit to the original [authors](#) and repository: <https://github.com/NanoBiostructuresRG>